

The SDE Interview Rebuild

Rebuilding CS fundamentals from intuition → interview confidence

A mentor-style study guide, not a textbook and not a cheat sheet. Every concept is taught the way a good senior engineer would explain it to you across a whiteboard: **what it is, why it was invented, the intuition, an analogy, how it actually works inside, examples, the interview questions, the traps, and how it connects to everything else.**

OOP

DBMS

Operating Systems

Computer Networks

Software Engineering

Git

Docker

System Design (light)

Designed as a **primary** study resource — read it front to back once, then use the revision section and question bank to drill. Excludes DSA, competitive programming, ML/Data Science, and project write-ups by design.

READ THIS FIRST

How to use this guide

This guide is built for **active** studying. Passive reading is the reason people forget fundamentals. Here's the machinery baked into every chapter and how to use it.

The teaching order (why it feels different)

Each anchor concept is taught in the same sequence so your brain learns to *reconstruct* ideas instead of memorizing them:

What is it? → Why invented / what problem? → Intuition → Analogy
 → How it works inside → Simple example → Technical example
 → Code → Interview Qs → Traps → Connections

The colored boxes

Box	Meaning	How to use it
Why (amber)	The problem the concept solves	If you only remember one thing, remember the "why"
Analogy (green)	Real-world mental picture	Your recall anchor under pressure
How it works (blue)	The internal mechanism	This is what senior interviewers probe
Mental Model (purple)	Simplest correct picture	Rebuild the concept from this
Memory Hook (navy)	One-sentence essence	Say it out loud
Trap (red)	Misconceptions that expose you	Interviewers love these
Connects to (teal)	Links to other topics	Shows senior-level understanding
Quiz (dashed yellow)	Active recall prompt	Answer <i>before</i> reading on; answers at chapter end

The depth tags

Concepts are tagged so you can triage your time:

FOUNDATION must understand **INTERVIEW** should explain confidently **DEEPER** stronger interviews only

The rule

THE ONE RULE

Never read a Quiz box's answer before attempting it. Recall failure is how memory forms. If you can explain a concept aloud without the page, you own it.

Table of Contents

- 1** Object-Oriented Programming (OOP).....
- 2** Database Management Systems (DBMS).....
- 3** Operating Systems (OS).....
- 4** Computer Networks.....
- 5** Software Engineering / SDE Fundamentals.....
- 6** Git — Version Control.....
- 7** Docker — Containers.....
- 8** Cross-Concept Connections — How It All Fits.....
- 9** Final Revision: Mind Maps, Revision Plans, Question Bank & Mock Test.....

SECTION 1

Object-Oriented Programming

OOP is not "classes and objects." OOP is a strategy for **managing complexity** as programs grow. Every OOP feature exists to fight one enemy: **a codebase that becomes impossible to change safely**. Keep that lens and OOP stops being a list of definitions.

1.1 Why OOP exists at all FOUNDATION

THE PROBLEM: PROCEDURAL CODE DOESN'T SCALE WITH CHANGE

In procedural programming you have **data** (variables, structs) and **functions** that operate on that data, living separately. For a small script this is perfect. The trouble starts when the program grows.

WHY OOP WAS INVENTED

As programs grew to millions of lines, the real cost stopped being "writing code" and became "changing code without breaking something far away." Procedural code spreads the rules about a piece of data across dozens of functions. Change the data's shape and you must hunt down every function that touches it. OOP's core move: **put the data and the functions that are allowed to touch it in the same box**, so a change stays local.

Concrete example. Imagine a bank account represented procedurally:

```
balance = 1000           // just a number, anywhere in the program

// Rule: balance must never go negative.
// But this rule lives only in your head. Any of these is legal code:
balance = balance - 5000   // oops, now -4000
balance = "hello"         // oops, wrong type
withdraw(balance, 5000)   // did withdraw() check the rule? who knows
transfer(balance, ...)    // did transfer() check it too? separately?
```

The rule "balance can't go negative" must be re-implemented in every function. Miss one and you have a bug. Now the same idea with OOP:

```
class Account {
  private double balance;           // no one outside can touch this directly

  public void withdraw(double amt) {
    if (amt > balance)              // the rule lives in ONE place
      throw new Error("Insufficient funds");
    balance -= amt;
  }
}
```

REAL-WORLD ANALOGY

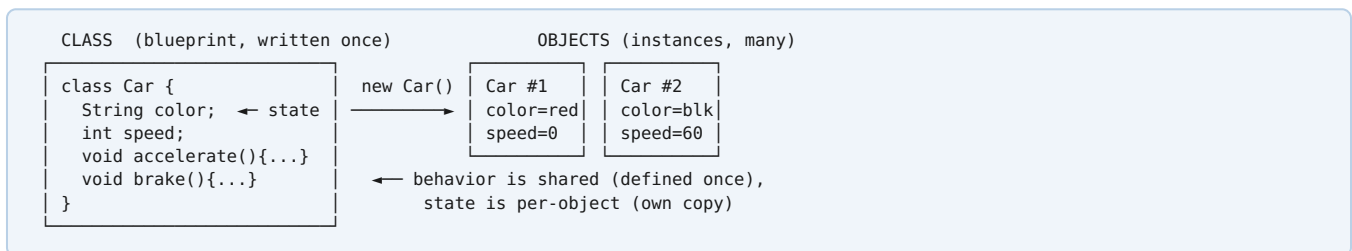
Procedural code is a workshop where the tools and the materials are scattered on separate tables and anyone can grab anything. OOP is giving each craftsman a **closed toolbox**: the materials and the only tools allowed to shape them are locked together, and you interact through a small hatch, not by reaching inside.

Aspect	Procedural	Object-Oriented
Organizing unit	Functions	Objects (data + behavior together)
Data + logic	Separated	Bundled
Data access	Usually global / passed around	Guarded by the object
Reuse mechanism	Call functions	Inheritance, composition, polymorphism
Best for	Small scripts, linear tasks	Large evolving systems, many collaborators
Main weakness	Change ripples everywhere	Can be over-engineered

MENTAL MODEL

An **object** is a tiny machine that holds some private state and exposes a few buttons (methods). You don't reach inside the machine; you press its buttons. A **class** is the blueprint the factory uses to stamp out those machines.

1.2 Class, Object, State, Behavior FOUNDATION



- **Class** = the definition / blueprint. Exists at compile time. Costs no memory for state by itself.
- **Object (instance)** = a concrete thing built from the class, living in memory at runtime.
- **State** = the object's data *right now* (the fields/attributes). Each object has its own.
- **Behavior** = what the object can do (methods). Defined once in the class, shared by all instances.

TRAP

"A class is an object." No. A class is a *type*; an object is a *value* of that type. "Dog" is a concept; the specific dog barking at you is the object. Interviewers ask this to check you're not just parroting terms.

1.3 Constructor & Destructor FOUNDATION

WHY CONSTRUCTORS EXIST

An object with half-initialized state is a landmine. A constructor guarantees that the moment an object exists, it is in a **valid** state — you can't create an `Account` without giving it required data. It removes the "did someone remember to initialize this?" class of bugs.

```

class Account {
  private double balance;
  private String owner;

  // Constructor: same name as class, no return type. Runs once, at creation.
  public Account(String owner, double opening) {
    if (opening < 0) throw new Error("Opening balance cannot be negative");
    this.owner = owner;
    this.balance = opening;
  }
}

Account a = new Account("Asha", 500); // constructor runs here

```

DESTRUCTOR / CLEANUP

A **destructor** runs when an object is destroyed, to release resources (files, sockets, memory). This matters most in languages with **manual/deterministic** memory management.

Language	Cleanup mechanism	When it runs
C++	Destructor <code>~Class()</code>	Deterministically, when object leaves scope / <code>delete</code>
Java	No real destructor; Garbage Collector + <code>AutoCloseable</code> / <code>try-with-resources</code>	GC runs "sometime later"; you can't rely on timing
Python	<code>__del__</code> (unreliable) + context managers (<code>with</code>)	When refcount hits 0, usually

TRAP

"Java has destructors called by `finalize()` ." `finalize()` exists but is deprecated and gives **no timing guarantee** — it may never run. Correct answer: Java uses garbage collection for memory and explicit resource management (`try-with-resources`) for files/sockets.

QUIZ 1.A

Predict: in C++, if you create a local object inside a function, when exactly does its destructor run? (Answer at chapter end.)

1.4 The Four Pillars

These are the four ideas interviewers expect you to explain *with reasons*, not slogans.

Pillar 1 — Encapsulation FOUNDATION

What: bundling data with the methods that operate on it, and **controlling access** to that data (usually making fields private, exposing behavior through methods).

WHY — WHAT BREAKS WITHOUT IT

Without encapsulation, any code anywhere can set your object into an invalid state. Every invariant ("balance ≥ 0 ", "email is valid", "list index in range") becomes everyone's responsibility, which means no one's. Encapsulation makes the object the **single guardian** of its own rules.

ANALOGY

An ATM. You can deposit and withdraw (public methods) but you cannot open the machine and rearrange the cash (private state). The machine enforces the rules regardless of who's using it.

```
// WITHOUT encapsulation
class Account {
    public double balance;
}
account.balance = -9999; // legal!

// WITH encapsulation
class Account {
    private double balance;
    public void deposit(double a){
        if(a<=0) throw ...;
        balance += a;
    }
    public double getBalance(){ return balance; }
}
```

HOW IT WORKS

Access modifiers (`private` , `protected` , `public`) are enforced by the compiler. The data still physically sits in memory; encapsulation is a *compile-time contract* about who may touch it, not a runtime lock. Getters/setters let you add validation, logging, or lazy computation later **without changing callers**.

TRAP

"Encapsulation = getters and setters on every field." Wrong and harmful. Blindly wrapping every field in `getX/setX` just re-exposes the data. Real encapsulation exposes *behavior* (`withdraw()`), hides *data*, and may have no setter at all.

Pillar 2 — Abstraction FOUNDATION

What: exposing *what* an object does while hiding *how* it does it. You program against a simple surface and ignore the messy internals.

WHY

Humans can't hold a million details in their head. Abstraction lets you use a component knowing only its contract. You call `list.sort()` without knowing (or caring) whether it's quicksort or timsort. The complexity still exists — it's just **hidden behind a clean interface**.

ANALOGY

Driving a car: the interface is steering wheel + pedals. You have no idea how fuel injection works, and you don't need to. If the engine is swapped for electric, your interface (steering, pedals) is unchanged.

ENCAPSULATION VS ABSTRACTION (THE CLASSIC CONFUSION)

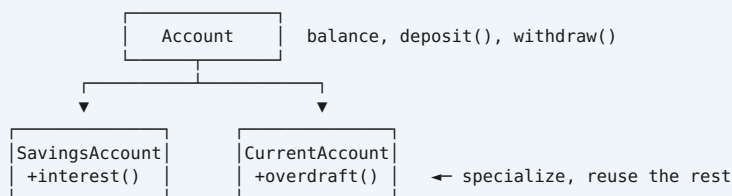
Abstraction = design-level: *hiding complexity*, deciding what to expose (the "what"). **Encapsulation** = implementation-level: *hiding data*, protecting state (the "how you protect it"). Abstraction is about the outside view; encapsulation is the mechanism that enforces it. They usually appear together but answer different questions.

Pillar 3 — Inheritance INTERVIEW

What: a class (child/subclass) acquires the fields and methods of another class (parent/superclass) and can add to or override them. Models an **"is-a"** relationship.

WHY

To reuse and specialize. A `SavingsAccount` *is an* `Account` with extra interest logic; you don't rewrite deposit/withdraw. It also enables polymorphism (next pillar) by giving unrelated subclasses a common type.

**TRAP: INHERITANCE IS OVERUSED**

Only use inheritance for a true "is-a". A `Stack` is NOT a `Vector` just because it's convenient to reuse code — that famous Java mistake leaks methods (`get(i)`) that break the stack. **Rule of thumb: prefer composition over inheritance.** Ask "is-a or has-a?" A car *has-an* engine (composition); a car *is-a* vehicle (inheritance).

QUIZ 1.B

Should `Square` inherit from `Rectangle` ? Think about `setWidth()` / `setHeight()` before answering. (Answer at chapter end — this is the Liskov trap.)

Pillar 4 — Polymorphism INTERVIEW

What: "many forms" — the same call behaves differently depending on the actual object. One interface, many implementations.

WHY

So calling code doesn't need a giant `if/else` on type. You write `shape.area()` once; each shape computes its own area. Add a new shape later and the calling code **doesn't change**. This is the engine behind extensible, plugin-style design.

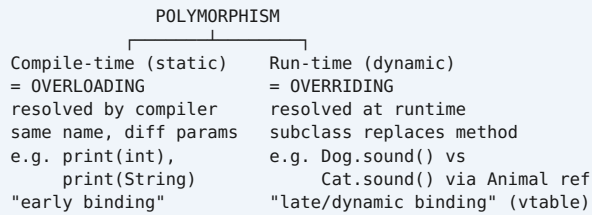
ANALOGY

A power socket. You plug in a phone charger, a lamp, a laptop — same socket interface (`plugIn()`), completely different behavior behind each plug.

```

Shape[] shapes = { new Circle(), new Square(), new Triangle() };
for (Shape s : shapes)
    print(s.area());    // each computes differently – no if/else on type
  
```

TWO KINDS OF POLYMORPHISM — KNOW THE SPLIT COLD



	Overloading (compile-time)	Overriding (run-time)
Definition	Same method name, different parameters , in the same class	Subclass provides new body for an inherited method, same signature
Resolved	At compile time (by argument types)	At run time (by actual object type)
Binding	Static / early	Dynamic / late (virtual dispatch)
Needs inheritance?	No	Yes
Return type can differ?	Not by itself (params must differ)	Yes (covariant returns allowed)

```
// OVERLOADING – same name, different signatures (compile-time)
int add(int a, int b)      { return a+b; }
double add(double a, double b) { return a+b; }
int add(int a, int b, int c) { return a+b+c; }

// OVERRIDING – subclass changes behavior (run-time)
class Animal { String sound(){ return "..."; } }
class Dog extends Animal { @Override String sound(){ return "Woof"; } }

Animal a = new Dog();
a.sound(); // "Woof" – decided at RUNTIME by the real object (Dog), not the ref type (Animal)
```

HOW RUNTIME DISPATCH WORKS INTERNALLY — THE VTABLE

For classes with overridable (virtual) methods, the compiler builds a **virtual method table (vtable)**: an array of function pointers, one per virtual method. Each object stores a hidden pointer (`vptr`) to its class's vtable. When you call `a.sound()`, the program follows `a`'s `vptr` → the vtable → the correct `sound()` for the real type. That extra pointer hop is why virtual calls are slightly slower and can't always be inlined.

```
Animal a = new Dog();

a → [ Dog object ]
    | vptr → [ Dog vtable ]
    | fields
              sound() → Dog::sound ← runtime picks this
              eat()   → Animal::eat
```

VIRTUAL FUNCTIONS, ABSTRACT CLASSES, INTERFACES

- **Virtual function** INTERVIEW: a method that *can* be overridden and is dispatched dynamically. In C++ you mark it `virtual`; in Java **every non-static, non-final method is virtual by default**; in Python all methods are virtual.
- **Abstract class**: a class you cannot instantiate; it may have some implemented methods and some **abstract** (unimplemented) methods that subclasses must fill in. Use it to share code + force a contract.
- **Interface**: a pure contract — a list of method signatures with (traditionally) no implementation and no state. A class can implement many interfaces (solving the "multiple inheritance" problem safely).

	Abstract class	Interface
Purpose	Share code + partial implementation	Define a pure capability/contract
State (fields)	Can have fields	Traditionally none (constants only)
Multiple inheritance	One base class only (in Java/C#)	Implement many
Relationship	"is-a"	"can-do" / "behaves-as"
Use when	Subclasses share real behavior	Unrelated classes share a capability

TRAP

"Interface vs abstract class — which is better?" There's no "better." Use an **abstract class** when subclasses genuinely share implementation and are the same kind of thing; use an **interface** when you want to declare a capability that even unrelated classes can provide (e.g. `Comparable`, `Serializable`). Modern Java interfaces can have `default` methods, blurring the line — mention this to sound current.

QUIZ 1.C

True/False: In Java, marking a method `private` makes it impossible to override. Explain. (Answer at end.)

1.5 Object Relationships: is-a vs has-a INTERVIEW

ASSOCIATION — "uses / knows" Teacher — Student (either can exist alone)

├ AGGREGATION — "has-a" (weak) Team ◊— Player
├ Players survive if Team is deleted

└ COMPOSITION — "has-a" (strong) House ◆— Room
 Rooms die with the House (owned)

INHERITANCE — "is-a" Dog → Animal

Relationship	Meaning	Lifetime	Example
Association	One object uses/knows another	Independent	Driver ↔ Car
Aggregation (weak has-a)	Whole contains parts, but parts can live alone	Independent	Team has Players
Composition (strong has-a)	Whole owns parts; parts die with whole	Dependent	House has Rooms; Order has OrderLines
Inheritance (is-a)	Subtype of another	N/A	Car is a Vehicle

MEMORY HOOK

*is-a → inherit. has-a → compose. When in doubt, **has-a**.*

1.6 The mechanics: static, this, access, copies, lifetime INTERVIEW

STATIC — BELONGS TO THE CLASS, NOT THE OBJECT

A `static` field/method belongs to the **class itself**, shared across all instances (one copy total). A static method can't use `this` because there's no particular object.

```
class Counter {
    static int total = 0;    // ONE copy shared by all Counters
    int id;
    Counter() { id = ++total; } // total is shared; id is per-object
}
new Counter(); new Counter(); // total == 2
```

THIS — THE CURRENT OBJECT

`this` is a hidden reference to the object the method was called on. Used to disambiguate fields from parameters (`this.balance = balance`) and to pass the object itself.

ACCESS MODIFIERS

Modifier	Visible to
<code>private</code>	Same class only
<code>protected</code>	Same class + subclasses (+ package in Java)
package/default	Same package (Java)
<code>public</code>	Everyone

COPY CONSTRUCTOR, SHALLOW VS DEEP COPY INTERVIEW

WHY THIS MATTERS

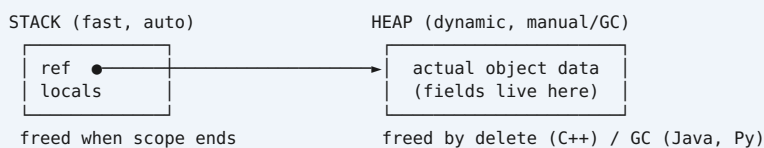
When you copy an object that contains references to other objects, do you copy the *references* (both point to the same inner object) or the *inner objects too*? Get this wrong and mutating one "copy" silently mutates the other — a classic hard-to-find bug.

```
Original → [ name:"A", list:● ]
SHALLOW copy → [ name:"A", list:● ] ← SAME list! change one, both change
DEEP copy → [ name:"A", list:● → [new independent list] ] ← safe
```

```
// Shallow: copies the top-level fields; inner references are shared
// Deep: recursively copies inner objects too, so they're independent
```

	Shallow copy	Deep copy
Copies	Top-level fields only	Everything, recursively
Nested objects	Shared (same reference)	Duplicated (independent)
Cost	Cheap	Expensive
Risk	Hidden shared-state bugs	Safe but slower

OBJECT LIFETIME & MEMORY



In Java/Python, objects live on the **heap**; variables are references on the **stack**. Object dies when nothing references it (GC reclaims it). In C++ you choose stack (auto-destroyed) or heap (`new` / `delete`).

QUIZ 1.D

You do `b = a` for two object variables in Java, then call `b.setName("X")`. Does `a`'s name change? Why? (Answer at end.)

1.7 SOLID Principles INTERVIEW DEEPER

Five design guidelines for classes that stay changeable. Interviewers ask you to name them *and give a one-line example*.

Letter	Principle	Plain meaning	Smell it fixes
S	Single Responsibility	A class should have one reason to change	God classes doing everything
O	Open/Closed	Open to extension, closed to modification	Editing old code for every new case
L	Liskov Substitution	A subtype must be usable wherever its base is	Subclass breaks base's contract (Square/Rectangle)
I	Interface Segregation	Many small interfaces > one fat one	Classes forced to implement methods they don't use
D	Dependency Inversion	Depend on abstractions, not concretions	High-level code nailed to a specific DB/vendor

ANALOGY FOR OPEN/CLOSED & DEPENDENCY INVERSION

A game controller port. You add new controllers (extension) without rewiring the console (no modification). The console depends on the *port shape* (abstraction), not on any specific controller (concretion).

```
// Dependency Inversion – depend on an interface, inject the concrete thing
interface PaymentGateway { void pay(int cents); }
class StripeGateway implements PaymentGateway { public void pay(int c){ /*...*/ } }

class Checkout {
    private final PaymentGateway gateway;           // depends on ABSTRACTION
    Checkout(PaymentGateway g){ this.gateway = g; } // injected – swap Stripe→PayPal freely
}
```

CONNECTS TO

SOLID reappears everywhere: **Dependency Inversion** is the basis of dependency injection frameworks (Spring) → Section 5. **Open/Closed** is exactly what **polymorphism** enables. **Interface Segregation** is why REST APIs are split into focused endpoints. The whole point is the same "make change safe" goal from 1.1.

1.8 Confusion Pairs

Pair	Key difference
Class vs Object	Blueprint vs a built instance
Overloading vs Overriding	Same name/diff params (compile-time) vs redefine inherited method (run-time)
Abstraction vs Encapsulation	Hide complexity (design) vs hide/protect data (mechanism)
Interface vs Abstract class	Pure contract, multiple vs shared code, single
Composition vs Inheritance	has-a (flexible) vs is-a (tight coupling)
Aggregation vs Composition	Parts survive whole vs parts die with whole
Shallow vs Deep copy	Share nested refs vs duplicate nested objects
static vs instance member	One per class vs one per object

1.9 Chapter Interview Kit

RAPID RECALL (SAY THE ANSWER IN ONE LINE)

1. Define encapsulation in terms of the problem it solves.
2. Difference between a class and an object?
3. What are the four pillars?
4. Overloading vs overriding?
5. What makes runtime polymorphism possible internally?
6. Abstract class vs interface — one deciding factor?
7. is-a vs has-a — which maps to inheritance?
8. What does `static` mean?
9. Shallow vs deep copy in one sentence?

10. What does the "L" in SOLID prevent?

INTERVIEW QUESTIONS (BE READY TO GO 2-3 MINUTES EACH)

1. Why prefer composition over inheritance? Give a real example.
2. Explain the vtable and how a virtual call is dispatched.
3. What happens if you override `equals()` but not `hashCode()`? (Java hint: breaks hash-based collections.)
4. Can a constructor be private? What is that used for? (Singletons, factories.)
5. How do you achieve multiple inheritance safely?
6. Walk through each SOLID letter with a one-line example.
7. What is the diamond problem and how do languages resolve it?
8. When would you make a class `final` / `sealed`?
9. Difference between association, aggregation, composition.
10. How does polymorphism enable the Open/Closed principle?

DEEP QUESTIONS (TEST UNDERSTANDING, NOT MEMORY)

1. Encapsulation still allows me to read/modify state through methods. So how is it "protection"? (Answer: it centralizes and validates access, controlling *how* not *whether*.)
2. Why is every Java method virtual by default while C++ requires `virtual`? What's the trade-off? (Safety/simplicity vs performance/explicitness.)
3. If interfaces now allow default methods, what still distinguishes them from abstract classes? (State + single-inheritance + intent.)
4. Give a case where inheritance is the wrong tool even though "it works."
5. Why can overloading be resolved at compile time but overriding cannot?

TRAP QUESTIONS

1. "Is method hiding (static methods) the same as overriding?" (No — static methods are hidden, resolved by reference type, not overridden.)
2. "Does adding getters/setters achieve encapsulation?" (Not automatically.)
3. "Square is-a Rectangle, so inheritance is correct, right?" (Violates Liskov.)
4. "Constructors can be inherited." (No — they're not inherited; they can be chained via `super()`.)
5. "Abstract classes can't have constructors." (They can — called via subclass.)

EXPLAIN-IT-ALoud

- Explain the difference between a class and an object to a non-programmer, then to a junior dev.
- Explain runtime polymorphism using the vtable, drawing the pointer hops.
- Explain why "prefer composition over inheritance" using Stack/Vector or Square/Rectangle.
- Explain all five SOLID principles in under 90 seconds.

30-SECOND EXPLANATIONS (REPRODUCE THESE IN AN INTERVIEW)

OOP IN 30S

OOP organizes programs as objects that bundle data with the behavior allowed to touch it, so changes stay local as systems grow. Its four pillars — encapsulation (protect state), abstraction (hide complexity), inheritance (reuse via is-a), polymorphism (one interface, many behaviors) — all serve one goal: making large code safe to change.

POLYMORPHISM IN 30S

Polymorphism lets the same call do different things based on the real object. Compile-time (overloading) picks a method by parameter types; run-time (overriding) picks a subclass method via a vtable lookup at execution. It replaces type-checking if/else chains and is what makes code extensible.

ONE PICTURE FOR THE WHOLE CHAPTER

Objects are little machines with private guts and public buttons. Encapsulation locks the guts, abstraction simplifies the buttons, inheritance lets you build specialized machines from general ones, and polymorphism lets one button do the right thing for whatever machine you're actually holding.

Chapter 1 — Quiz Answers

ANSWERS

1.A The destructor runs deterministically when the local object goes out of scope — at the closing `}` of the block (or on stack unwinding during an exception), in reverse order of construction.

1.B (Square/Rectangle) No. If `Rectangle` has independent `setWidth` / `setHeight`, then a `Square` that keeps sides equal violates the base contract: code expecting "set width to 5, height stays" breaks. This is the Liskov Substitution violation. Prefer composition or a shared `Shape` abstraction.

1.C True. A `private` method isn't visible to subclasses, so a subclass method with the same name is a *new* method, not an override. Overriding requires the method to be visible and virtual.

1.D Yes, `a`'s name changes. `b = a` copies the *reference*, not the object; both point to the same object on the heap. To get independent objects you need a copy constructor / clone.

SECTION 2

Database Management Systems

A database is not "where you store data." It's a system that lets many users read and write shared data **concurrently, correctly, durably, and fast** — four guarantees that are shockingly hard to get right yourself. Every DBMS feature exists to give you one of those guarantees.

2.1 From data to databases — why not just files? FOUNDATION

- **Data** = raw facts (the number 500, the string "Asha").
- **Information** = data with meaning ("Asha's balance is 500").
- **Database** = an organized collection of related data.
- **DBMS** = the software that manages that data (stores, queries, protects, coordinates access).

WHY NOT STORE EVERYTHING IN PLAIN FILES?

Try building a bank on text files and you hit walls fast: (1) **No structure/querying** — "sum all balances in Mumbai" means writing a parser. (2) **No concurrency** — two people editing the file at once corrupt it. (3) **No integrity** — nothing stops a negative balance or an orphan record. (4) **No crash safety** — power dies mid-write, file is half-updated. (5) **Redundancy** — the same customer's address copied into 50 files, now inconsistent. A DBMS solves all five.

Problem with files	How a DBMS solves it
Ad-hoc querying	SQL query engine
Concurrent access corruption	Transactions + locking
Invalid data	Constraints (types, keys, checks)
Crash mid-write	Durability + write-ahead log (WAL)
Duplicated, inconsistent data	Normalization + relationships
Slow lookups	Indexes

DBMS VS RDBMS

DBMS = any data-managing software. **RDBMS** (Relational DBMS) = a DBMS that organizes data into **tables (relations)** with rows and columns and enforces relationships between them via keys (MySQL, PostgreSQL, Oracle, SQL Server).

TABLE: Customers

id	name	city
1	Asha	Mumbai
2	Ben	Delhi

row = record (one customer)
 column = attribute (a field)
 ← schema = the structure (names + types)
 ← a row / tuple

MENTAL MODEL

An RDBMS is a stack of spreadsheets (tables) that are wired together by shared ID columns (keys), guarded by a rulebook (constraints), and served by a query engine that can slice, join, and aggregate them safely while thousands of people edit at once.

2.2 SQL essentials FOUNDATION INTERVIEW

LOGICAL ORDER OF A SELECT (CRUCIAL MENTAL MODEL)

You *write* SQL in one order, but the engine *evaluates* it in another. This one diagram explains 80% of SQL confusion (like "why can't I use an alias in WHERE?").

WRITTEN order: SELECT – FROM – WHERE – GROUP BY – HAVING – ORDER BY

EXECUTION order:

1. FROM — pick tables, do joins
2. WHERE — filter individual rows (BEFORE grouping)
3. GROUP BY — collapse rows into groups
4. HAVING — filter the groups (AFTER grouping)
5. SELECT — choose/compute columns, aliases created HERE
6. DISTINCT — remove duplicate result rows
7. ORDER BY — sort
8. LIMIT — cut

TRAP: WHERE VS HAVING

WHERE filters rows before grouping and cannot use aggregate functions. **HAVING** filters *groups* after **GROUP BY** and is where aggregates like **COUNT()**, **SUM()** belong. "Salary > 50000" → WHERE. "Departments whose average salary > 50000" → HAVING.

```
SELECT city, COUNT(*) AS customers
FROM Customers
WHERE active = true           -- filter rows first
GROUP BY city                -- group by city
HAVING COUNT(*) > 10         -- keep only cities with >10 customers
ORDER BY customers DESC;
```

Clause	Does	Gotcha
SELECT	Chooses columns / computes values	Aliases defined here can't be used in WHERE
WHERE	Filters rows	No aggregates allowed
GROUP BY	Collapses rows into groups	Non-aggregated SELECT cols must be in GROUP BY
HAVING	Filters groups	Runs after grouping
ORDER BY	Sorts output	Can use SELECT aliases
DISTINCT	Removes duplicate rows	Applies to the whole row selected

AGGREGATE FUNCTIONS

COUNT, **SUM**, **AVG**, **MIN**, **MAX**. They collapse many rows into one value. **COUNT(*)** counts rows; **COUNT(col)** ignores NULLs — a common trap.

SUBQUERIES AND CTES

A **subquery** is a query inside another. A **CTE (Common Table Expression)** is a named temporary result defined with **WITH**, which makes complex queries readable and can be recursive.

```
-- Subquery: customers who spent above the overall average
SELECT name FROM Customers
WHERE id IN (SELECT customer_id FROM Orders
            GROUP BY customer_id
            HAVING SUM(amount) > (SELECT AVG(amount) FROM Orders));

-- Same idea, readable, with a CTE
WITH spend AS (
  SELECT customer_id, SUM(amount) AS total
  FROM Orders GROUP BY customer_id
)
SELECT c.name, s.total
FROM spend s JOIN Customers c ON c.id = s.customer_id
WHERE s.total > (SELECT AVG(total) FROM spend);
```

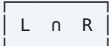
CASE (INLINE CONDITIONAL)

```
SELECT name,
       CASE WHEN balance >= 100000 THEN 'Premium'
            WHEN balance >= 10000  THEN 'Standard'
            ELSE 'Basic' END AS tier
FROM Accounts;
```

Joins — visually INTERVIEW

Joins combine rows from two tables based on a related column. This is *the* most-tested SQL topic.

Left = Customers	Right = Orders
A	(A → 2 orders)
B	(B → 1 order)
C (no orders)	X (order with no matching customer)

INNER JOIN only matches	LEFT JOIN all Left + matches	RIGHT JOIN all Right+matches	FULL OUTER JOIN everything
			
A, B	A, B, + C (nulls)	A, B, + X (nulls)	A, B, C, X (nulls both sides)

Join	Returns	Use when
INNER	Only rows matching in both	You want records that exist on both sides
LEFT (OUTER)	All left rows; right cols NULL if no match	"All customers, even those with no orders"
RIGHT (OUTER)	All right rows; left NULL if no match	Mirror of LEFT (rarely used)
FULL OUTER	All rows from both; NULLs where no match	Reconcile two sets, find gaps either side
SELF JOIN	Table joined to itself (aliased twice)	Hierarchies: employee → manager
CROSS JOIN	Cartesian product (every combo)	Rarely on purpose; often an accidental bug

```
-- SELF JOIN: each employee with their manager's name
SELECT e.name AS employee, m.name AS manager
FROM   Employees e
LEFT JOIN Employees m ON e.manager_id = m.id;
```

TRAP

"LEFT JOIN then filter the right table in WHERE." Putting `WHERE orders.status='X'` after a LEFT JOIN silently turns it into an INNER JOIN (NULL rows fail the condition). Put such conditions in the `ON` clause instead if you want to keep unmatched left rows.

QUIZ 2.A

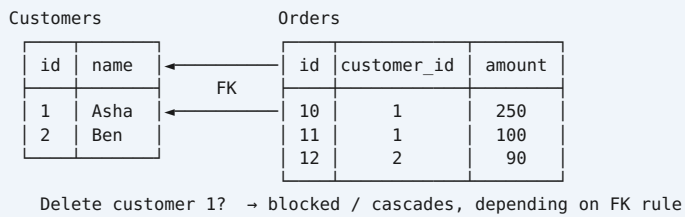
Table A has 5 rows, table B has 3 rows, no matching keys. How many rows does `A CROSS JOIN B` return? And `A INNER JOIN B ON ...` with no matches? (Answers at end.)

2.3 Keys & Integrity FOUNDATION

Key	Definition	Example
Candidate key	Any minimal set of columns that uniquely identifies a row	<code>email</code> , or <code>ssn</code>
Primary key	The chosen candidate key; unique + NOT NULL; one per table	<code>id</code>
Composite key	A key made of multiple columns together	(<code>order_id</code> , <code>product_id</code>)
Foreign key	A column referencing another table's primary key	<code>Orders.customer_id</code> → <code>Customers.id</code>
Unique key	Enforces uniqueness but may allow one NULL; can be many per table	<code>username</code>

WHY FOREIGN KEYS / REFERENTIAL INTEGRITY

They stop **orphan records**. Without a foreign key, you could insert an order for customer #999 who doesn't exist, or delete a customer while their orders still point to them. Referential integrity means: a foreign key value must always match an existing primary key (or be NULL). The DBMS enforces this on every insert/update/delete.



Common **constraints**: **NOT NULL**, **UNIQUE**, **PRIMARY KEY**, **FOREIGN KEY**, **CHECK (balance >= 0)**, **DEFAULT**.

2.4 Normalization INTERVIEW

WHY NORMALIZE

Storing data redundantly causes **anomalies**: **update anomaly** (change an address in one row, forget another → inconsistent), **insertion anomaly** (can't add a product without an order), **deletion anomaly** (delete the last order and lose the customer's info). Normalization removes redundancy by splitting data into related tables so each fact is stored **once**.

FUNCTIONAL DEPENDENCY (THE FOUNDATION)

$A \rightarrow B$ means "A determines B": if you know A, B is fixed. $student_id \rightarrow student_name$. Normalization is about arranging tables so every non-key column depends on **the key, the whole key, and nothing but the key**.

1NF — ATOMIC VALUES, NO REPEATING GROUPS

BAD (violates 1NF: multiple phones in one cell)

id	name	phones
1	Asha	111, 222, 333

← not atomic

FIXED (each value atomic, one per row)

id	name	phone
1	Asha	111
1	Asha	222

2NF — NO PARTIAL DEPENDENCY ON PART OF A COMPOSITE KEY

BAD: key is (order_id, product_id), but product_name depends on product_id ALONE

order_id	product_id	product_name	qty
10	P1	Keyboard	2
11	P1	Keyboard	1

← product_name repeats, partial dependency

FIXED: split out Products

OrderItems(order_id, product_id, qty) Products(product_id, product_name)

3NF — NO TRANSITIVE DEPENDENCY (NON-KEY → NON-KEY)

BAD: $emp_id \rightarrow dept_id \rightarrow dept_name$ (dept_name depends on a NON-key column)

emp_id	dept_id	dept_name
--------	---------	-----------

← change dept_name → many rows

FIXED:

Employees(emp_id, dept_id) Departments(dept_id, dept_name)

BCNF — STRICTER 3NF

Every determinant (left side of a functional dependency) must be a candidate key. BCNF handles rare cases 3NF misses where a non-key column determines part of a candidate key. For interviews: know that **BCNF is a stronger form of 3NF** and give the "every determinant is a candidate key" line.

Form	Rule (plain)	Removes
1NF	Atomic values, no repeating groups	Multi-valued cells
2NF	1NF + no partial dependency on part of a composite key	Partial dependencies
3NF	2NF + no transitive (non-key → non-key) dependency	Transitive dependencies
BCNF	3NF + every determinant is a candidate key	Remaining anomalies

MEMORY HOOK (THE FAMOUS LINE)

Every non-key column must depend on **the key (1NF/relation), the whole key (2NF), and nothing but the key (3NF)** — "so help me Codd."

TRAP: NORMALIZATION IS NOT ALWAYS THE GOAL

Over-normalizing means lots of joins, which can be slow. Analytics/read-heavy systems often **denormalize** deliberately (duplicate data for speed). Interview-smart answer: "Normalize for integrity in transactional (OLTP) systems; denormalize selectively for read performance in reporting (OLAP) systems."

QUIZ 2.B

A table (student_id, course_id, course_name, grade) with key (student_id, course_id). Which normal form does it violate and why? (Answer at end.)

2.5 Transactions & ACID INTERVIEW

A **transaction** is a group of operations treated as a single unit: all succeed or none do.

ANALOGY

Transferring money: debit A, credit B. If the power dies after the debit but before the credit, money vanishes. A transaction guarantees both happen or neither — like a "commit or roll back" undo button.

```
BEGIN
  debit A -100
  credit B +100
COMMIT
```

} one atomic unit
(or ROLLBACK undoes everything)

ACID	Guarantee	What it prevents
A tomicity	All-or-nothing	Half-done transfers
C onsistency	DB moves valid state → valid state (constraints hold)	Broken invariants
I solation	Concurrent txns don't corrupt each other	Race conditions on data
D urability	Once committed, survives crashes	Losing confirmed data

HOW DURABILITY WORKS

Before changing the actual data files, the DBMS writes the change to a **write-ahead log (WAL)** on disk and flushes it. If the server crashes mid-operation, on restart it replays the log to finish committed transactions and undo incomplete ones. That's how "committed = safe" survives power loss.

CONCURRENCY PROBLEMS (WHAT ISOLATION PREVENTS)

Anomaly	What happens
Dirty read	You read data another transaction wrote but hasn't committed — it may roll back
Non-repeatable read	You read a row twice in one txn and get different values (someone updated it between)
Phantom read	You run the same query twice and get different sets of rows (someone inserted/deleted)
Lost update	Two txns read then write the same row; one overwrites the other's change

ISOLATION LEVELS (WEAKEST → STRONGEST)

Level	Dirty read	Non-repeatable	Phantom	Cost
READ UNCOMMITTED	Possible	Possible	Possible	Cheapest
READ COMMITTED	Prevented	Possible	Possible	Low
REPEATABLE READ	Prevented	Prevented	Possible*	Medium
SERIALIZABLE	Prevented	Prevented	Prevented	Highest

*In MySQL's InnoDB, REPEATABLE READ also blocks phantoms via gap locks; standards vary — mention the DB matters.

MEMORY HOOK

Higher isolation = more correctness, less concurrency. It's a dial between "fast but risky" and "safe but serialized."

LOCKS & DEADLOCKS

- **Shared (S) lock:** many readers can hold it at once; blocks writers. "Read lock."
- **Exclusive (X) lock:** one writer only; blocks everyone. "Write lock."
- **Deadlock:** T1 holds A waits for B; T2 holds B waits for A — both stuck forever. The DBMS detects the cycle and kills one (the "victim") to break it.

```
T1: lock(A) ✓ ... wants lock(B) ✗ (held by T2)
T2: lock(B) ✓ ... wants lock(A) ✗ (held by T1)    → DEADLOCK
                                         DBMS aborts one transaction
```

QUIZ 2.C

You need "the report shows the same rows if I run it twice within one transaction, and no new rows appear." Which isolation level? (Answer at end.)

2.6 Indexing INTERVIEW

WHY INDEXES EXIST

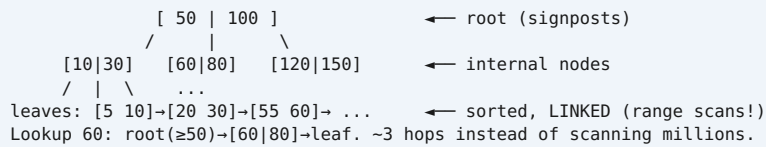
Without an index, finding rows means a **full table scan** — read every row. On a million-row table that's a million reads. An index is a pre-sorted lookup structure that turns "scan everything" into "jump straight there," like a book's index vs reading every page.

ANALOGY

A library. No index = walk every shelf to find a book. An index card catalog (sorted by title) lets you binary-search to the exact shelf in a few steps.

HOW: B+ TREES

Most relational indexes are **B+ trees**: balanced, wide trees where all data sits in sorted leaf nodes linked together, and internal nodes are just signposts. Lookups take $O(\log n)$ — a few disk reads even for billions of rows — and the linked leaves make **range queries** (`BETWEEN` , `ORDER BY`) fast.



	Clustered index	Non-clustered index
What	The table rows ARE stored in this index's order	Separate structure pointing to the rows
How many	One per table (data can be sorted one way)	Many
Speed	Fastest for range scans on the key	Extra hop to fetch the actual row
Usually	The primary key	Secondary indexes on other columns

TRAP: WHY NOT INDEX EVERY COLUMN?

Indexes speed up reads but **slow down writes** (every insert/update/delete must also update every index) and consume storage. They also help only certain query shapes. So you index columns used in **WHERE**, **JOIN**, and **ORDER BY** — not everything. Indexing is a read/write trade-off, not free speed.

WHEN AN INDEX DOES NOT HELP

(1) Low-cardinality columns (e.g. boolean **is_active**) — too many matches. (2) Functions on the column: **WHERE YEAR(created)=2024** can't use an index on **created** unless it's a functional index. (3) Leading wildcard **LIKE '%abc'**. (4) Small tables where a scan is already fast.

QUIZ 2.D

You have an index on (**last_name**, **first_name**). Does a query filtering only by **first_name** use it efficiently? (Answer at end — this is the "leftmost prefix" rule.)

2.7 SQL vs NoSQL, Scaling, Ops INTERVIEW DEEPER

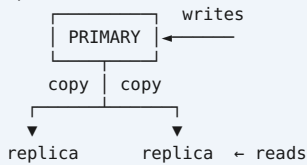
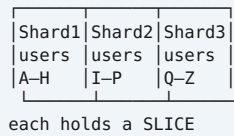
	SQL (Relational)	NoSQL
Data model	Tables, rows, fixed schema	Documents, key-value, wide-column, graph
Schema	Rigid, defined upfront	Flexible / schema-less
Relationships	Joins, foreign keys	Usually embedded/denormalized
Consistency	Strong (ACID)	Often eventual (BASE), tunable
Scaling	Vertical mainly; horizontal is hard	Horizontal (sharding) by design
Best for	Complex queries, transactions, integrity	Huge scale, flexible/varied data, high write throughput
Examples	PostgreSQL, MySQL	MongoDB (doc), Redis (KV), Cassandra (wide-col)

TRAP

"NoSQL is faster/better." Not inherently. NoSQL trades away joins and (often) strong consistency for scale and flexibility. Choose by workload: need multi-row transactions and complex joins → SQL; need to store varied documents at massive scale with simple access patterns → NoSQL. Many systems use both.

- **Replication:** keep copies of the DB on multiple servers. A **primary (leader)** takes writes; **replicas (followers)** serve reads and provide failover. Improves read scale + availability.
- **Sharding:** split one big dataset *horizontally* across servers by a shard key (e.g. **user_id % N**). Each shard holds part of the data. Enables datasets bigger than one machine, at the cost of cross-shard queries being hard.
- **Connection pooling:** opening a DB connection is expensive; a pool keeps a set of reusable open connections so requests borrow/return instead of reconnecting each time.
- **Query optimization basics:** use **EXPLAIN** to see the plan; add indexes for slow **WHERE / JOIN**; avoid **SELECT ***; filter early; beware N+1 queries (see Section 5).

Replication (scale reads, failover)

Sharding (scale data/writes)
requests → router

2.8 Practical SQL problems (with reasoning)

P1. Second-highest salary.

```
SELECT MAX(salary) FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);
-- or, robust to ties / Nth: use DENSE_RANK()
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rk FROM Employees
) t WHERE rk = 2;
```

P2. Find duplicates.

```
SELECT email, COUNT(*) FROM Users
GROUP BY email HAVING COUNT(*) > 1;
```

P3. Customers with no orders (anti-join).

```
SELECT c.name FROM Customers c
LEFT JOIN Orders o ON o.customer_id = c.id
WHERE o.id IS NULL; -- left rows that found no match
```

P4. Running total (window function).

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM Orders;
```

CONNECTS TO

Transactions/ACID are the DB-level version of **OS synchronization** (Section 3) — both fight race conditions with locks and can deadlock. Indexes are a **B+ tree**, a data structure. Replication/sharding/pooling reappear in **system design** (Section 5). The DB is the bottom of the request lifecycle: Client → API → ... → **DB query** → **transaction** → **index**.

2.9 Confusion Pairs

Pair	Difference
WHERE vs HAVING	Filter rows (pre-group) vs filter groups (post-group)
DELETE vs TRUNCATE vs DROP	DELETE removes rows (logged, WHERE, rollback-able) · TRUNCATE removes all rows fast (no WHERE, resets, minimal log) · DROP removes the whole table/structure
Primary vs Unique key	PK: one, NOT NULL · Unique: many, may allow a NULL
Clustered vs Non-clustered index	Rows stored in that order (one) vs separate pointer structure (many)
Inner vs Outer join	Only matches vs keep unmatched with NULLs
SQL vs NoSQL	Structured + ACID + joins vs flexible + horizontal scale
Normalization vs Denormalization	Remove redundancy (integrity) vs add redundancy (read speed)
Replication vs Sharding	Copy whole DB (reads/HA) vs split data (capacity)

2.10 Chapter Interview Kit

RAPID RECALL

1. Why not use files instead of a DB? (name 3 reasons)
2. Execution order of a SELECT?
3. WHERE vs HAVING?
4. Four join types in one line each?
5. What is a foreign key / referential integrity?
6. State ACID.
7. Three read anomalies?
8. Why is an index fast?
9. Why not index everything?
10. Clustered vs non-clustered?
11. Replication vs sharding?
12. DELETE vs TRUNCATE vs DROP?

INTERVIEW QUESTIONS

1. Walk me through normalizing a denormalized orders table to 3NF.
2. Explain how a B+ tree index makes lookups fast, with the tree.
3. What isolation level would you use for a banking transfer and why?
4. How do deadlocks happen and how does the DB resolve them?
5. Write a query for the Nth highest salary.
6. When would you denormalize on purpose?
7. What's the N+1 query problem and how do you fix it?
8. How does durability survive a crash? (WAL)
9. SQL vs NoSQL — how do you decide?
10. What's the leftmost-prefix rule for composite indexes?

DEEP QUESTIONS

1. Why can't WHERE use aggregate functions but HAVING can? (Execution order.)
2. REPEATABLE READ prevents non-repeatable reads but not always phantoms — why the difference? (Row locks vs range/gap locks.)
3. Why does adding indexes eventually make a write-heavy table slower?
4. Consistency in ACID vs "eventual consistency" in distributed NoSQL — same word, different meaning?
5. When does an index on a column get ignored by the optimizer?

TRAP QUESTIONS

1. "COUNT(*) and COUNT(column) are the same." (No — COUNT(column) skips NULLs.)
2. "A LEFT JOIN with a WHERE on the right table keeps unmatched rows." (No — becomes inner.)
3. "TRUNCATE can be rolled back like DELETE everywhere." (DB-dependent; often minimally logged.)
4. "More normalization is always better." (No — joins cost.)
5. "NoSQL has no schema so there's no design." (Access-pattern design is still critical.)

EXPLAIN-IT-ALoud

- Explain ACID to a junior dev using a money transfer.
- Explain why an index speeds reads but slows writes.
- Explain the difference between replication and sharding and when you'd use each.
- Normalize a messy table aloud, naming each anomaly you remove.

30-SECOND EXPLANATIONS

INDEXES IN 30S

An index is a sorted lookup structure (usually a B+ tree) that lets the DB jump to matching rows in $O(\log n)$ instead of scanning the whole table. It makes reads fast but every write must also update the index, and it costs storage — so you index columns used in filters, joins, and sorting, not all of them.

TRANSACTIONS IN 30S

A transaction groups operations so they're atomic (all or nothing), keep the DB consistent, run isolated from each other, and are durable once committed. Isolation levels dial how much concurrent transactions can interfere, trading correctness against throughput; locks enforce it and can deadlock, which the DB resolves by aborting one.

Chapter 2 — Quiz Answers

ANSWERS

2.A CROSS JOIN returns $5 \times 3 = 15$ rows (every combination). INNER JOIN with no matching keys returns 0 rows.

2.B It violates 2NF (and thus not fully normalized): the key is composite (`student_id, course_id`), but `course_name` depends only on `course_id` — a partial dependency. Fix: move `course_name` into a `Courses(course_id, course_name)` table.

2.C SERIALIZABLE (or MySQL InnoDB's REPEATABLE READ, which also blocks phantoms via gap locks). You need both non-repeatable reads and phantoms prevented.

2.D No. A composite index on (`last_name, first_name`) is sorted by `last_name` first, so it can only be used efficiently for queries that filter on the leftmost column(s). Filtering by `first_name` alone can't use it (leftmost-prefix rule) — you'd need a separate index.

SECTION 3

Operating Systems

The OS is the software that turns a pile of hardware into something programs can safely share. Its whole job is **illusion and arbitration**: give every program the illusion it owns the CPU and all of memory, while secretly juggling dozens of them and stopping any one from wrecking the others.

3.1 What an OS is and why FOUNDATION

WHY AN OS EXISTS

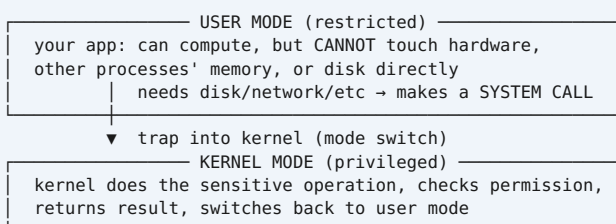
Without it, every program would have to talk to hardware directly, manage memory manually, and cooperate with every other program by trust alone — impossible. The OS is a **resource manager + protector**: it multiplexes the CPU, memory, disk, and devices among many programs, and enforces isolation so a bug in one can't corrupt another or the machine.

OS RESPONSIBILITIES

Area	What the OS does
Process management	Create, schedule, and terminate processes
Memory management	Allocate memory, virtual memory, protection
File system	Organize storage into files/directories
Device / I/O	Talk to hardware via drivers
Security / protection	Users, permissions, isolation

KERNEL, USER MODE VS KERNEL MODE

The **kernel** is the core of the OS that always runs and has full hardware access. To protect the machine, the CPU runs in two modes:



ANALOGY

User mode is a hotel guest; kernel mode is the staff. You can't walk into the kitchen or the safe. You ring the front desk (system call), staff (kernel) does the privileged action for you and hands back the result.

TRAP

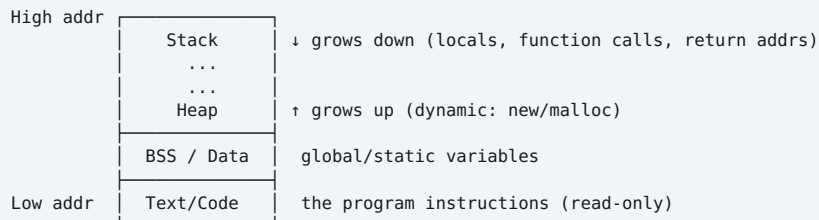
"A system call is just a function call." No — a normal call stays in user mode; a system call **switches CPU mode** into the kernel (a trap), which is far more expensive. That cost is why we batch I/O and avoid excessive syscalls.

3.2 Processes FOUNDATION INTERVIEW

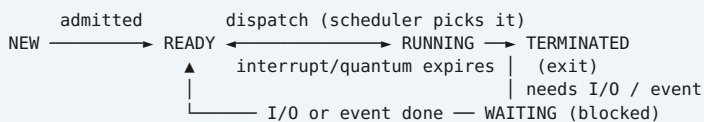
PROGRAM VS PROCESS

A **program** is a passive file on disk (instructions). A **process** is a program *in execution* — a live entity with its own memory, registers, and state. Same program run twice = two independent processes. (Program = recipe; process = actually cooking it, twice = two cooks.)

PROCESS MEMORY LAYOUT



PROCESS STATES



State	Meaning
New	Being created
Ready	Waiting for the CPU (able to run)
Running	Currently executing on a CPU
Waiting/Blocked	Waiting for I/O or an event
Terminated	Finished

PCB & CONTEXT SWITCHING

The **Process Control Block (PCB)** is the OS's record of a process: its PID, state, program counter, register values, memory maps, open files, priority. When the OS switches the CPU from process A to B, it does a **context switch**: save A's registers into A's PCB, load B's from B's PCB.

HOW CONTEXT SWITCHING WORKS & WHY IT'S NOT FREE

Saving and restoring registers is cheap-ish, but the switch also pollutes the CPU cache and (for a different process) the TLB, so the new process starts with cold caches. That overhead is "pure waste" — no user work happens during it — which is why switching too often (tiny time slices) hurts throughput.

PROCESS CREATION & SYSTEM CALLS

In Unix, a process creates a child with `fork()` (clone the caller) then often `exec()` (replace the clone's memory with a new program). System calls are the API to the kernel: `read`, `write`, `open`, `fork`, `exec`, `wait`, `exit`.

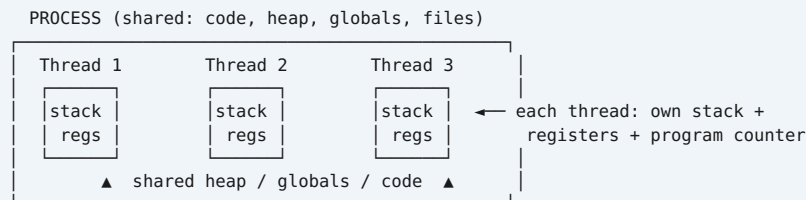
QUIZ 3.A

Two processes run the same program. Do they share the same variables in memory? Why or why not? (Answer at end.)

3.3 Threads INTERVIEW

WHY THREADS

A process is heavy: its own memory space, expensive to create and switch. But often you want *concurrency within one program* (a browser rendering while downloading). Threads are **lightweight execution paths inside one process** that share the process's memory — cheap to create, cheap to switch, and can run truly in parallel on multiple cores.



	Process	Thread
Memory	Own, isolated address space	Shares process memory
Creation cost	Heavy	Light
Context switch	Expensive (swap address space, TLB)	Cheaper (same address space)
Crash impact	Isolated — one crash doesn't kill others	A bad thread can crash the whole process
Communication	IPC (pipes, sockets, shared mem)	Shared variables (fast, but needs locks)

TRAP: THREADS SHARE MEMORY — THAT'S THE DANGER

Because threads share the heap and globals, two threads touching the same variable without coordination cause **race conditions** (next section). "Thread-safe" means correct when accessed by multiple threads at once. This is the #1 source of concurrency bugs.

User-level vs kernel-level threads: user-level threads are managed by a library in user space (fast switch, but the kernel sees one process — one blocking call blocks all). Kernel-level threads are scheduled by the OS (true parallelism, higher switch cost). Most modern runtimes map many user threads onto kernel threads.

MEMORY HOOK

Processes are isolated apartments; threads are roommates sharing one apartment — fast to coordinate, easy to step on each other.

3.4 CPU Scheduling INTERVIEW

WHY SCHEDULING

Many processes are ready; one CPU (or a few cores). The scheduler decides who runs next, balancing throughput, fairness, and responsiveness. Different algorithms optimize different things.

KEY METRICS

- **Turnaround time** = completion time – arrival time (total time in system).
- **Waiting time** = turnaround – burst time (time spent waiting, not running).
- **Response time** = first time it runs – arrival (matters for interactivity).

WORKED EXAMPLE (ALL ARRIVE AT TIME 0)

P1 burst=5 P2 burst=3 P3 burst=8

FCFS (First-Come-First-Served) — run in arrival order P1, P2, P3:

```
Gantt: | P1      | P2      | P3      |
        0        5        8        16
Completion: P1=5 P2=8 P3=16
Turnaround: 5,   8,   16
Waiting:    0,   5,   8   → avg waiting = 13/3 ≈ 4.33
```

SJF (Shortest Job First, non-preemptive) — run shortest burst first: P2(3), P1(5), P3(8):

```
Gantt: | P2      | P1      | P3      |
        0        3        8        16
Waiting: P2=0 P1=3 P3=8 → avg waiting = 11/3 ≈ 3.67 (better!)
```

Round Robin (time quantum = 2) — cycle giving each 2 units:

```
Gantt: |P1|P2|P3|P1|P2|P3|P1|P3|P3|
        2  4  6  8  9 11 12 14 16
Completion: P2=9 P1=12 P3=16
Waiting:    P1=7 P2=6 P3=8 → avg waiting = 7 (worse avg, but responsive)
```

Algorithm	Idea	Pros	Cons
FCFS	Queue order	Simple, fair-ish	Convoy effect (short jobs wait behind long)
SJF (non-preempt)	Shortest burst first	Optimal average waiting time	Needs to know burst; can starve long jobs
SRTF (preempt SJF)	Preempt if a shorter job arrives	Even better average waiting	More switches; starvation
Round Robin	Fixed time slice each	Great responsiveness, fair	Higher avg turnaround; quantum tuning
Priority	Highest priority first	Important work first	Starvation (fixed by aging)

TRAP

"Round Robin has the best average waiting time." No — SJF/SRTF minimize *average waiting time*. RR optimizes *responsiveness/fairness*. Also: too-small a quantum → too many context switches (overhead); too-large → RR degrades into FCFS.

QUIZ 3.B

What is "starvation" and which algorithm classically causes it? What's the standard fix? (Answer at end.)

3.5 Synchronization INTERVIEW

THE PROBLEM: RACE CONDITIONS

When two threads read-modify-write shared data concurrently, the result depends on timing and can be wrong. The classic: two threads each do `balance = balance + 100`. Each reads 1000, adds 100, writes 1100 — final is 1100, not 1200. One update is lost. The fix is to make such a section **mutually exclusive**.

Thread A	Thread B	balance
read balance(1000)		1000
	read balance(1000)	1000
add 100 → 1100		1000
	add 100 → 1100	1000
write 1100		1100
	write 1100	1100 ← LOST an update! should be 1200

- **Critical section:** the code that touches shared data and must not run concurrently.
- **Mutex (mutual exclusion lock):** a lock that only one thread holds at a time. Lock before the critical section, unlock after. Ownership: only the locker unlocks.
- **Semaphore:** a counter with `wait()` / `signal()`. A **binary** semaphore (0/1) acts like a lock; a **counting** semaphore allows N concurrent holders (e.g. a pool of 5 DB connections). No ownership concept.
- **Monitor:** a higher-level construct (e.g. Java `synchronized`) that bundles the lock with the data — the language enforces that you're inside the lock to touch the data.
- **Atomic operation:** an operation the hardware guarantees is indivisible (e.g. compare-and-swap), used to build lock-free counters.

MUTEX VS SEMAPHORE (CLASSIC CONFUSION)

A **mutex** is a locking mechanism with **ownership** — the thread that locks must unlock (used for mutual exclusion). A **semaphore** is a **signaling** mechanism — a counter, no ownership, used to control access to N resources or to signal between threads. "Mutex = one key to one bathroom; counting semaphore = N keys to a parking lot with N spots."

QUIZ 3.C

You have a pool of exactly 3 database connections and many threads wanting them. Mutex or semaphore? What value? (Answer at end.)

3.6 Deadlocks INTERVIEW

ANALOGY

Four cars reach a 4-way stop simultaneously, each waiting for the car on its right to move. Nobody moves. Ever. That's a deadlock — a cycle of waiting.

FOUR NECESSARY CONDITIONS (ALL MUST HOLD)

Condition	Meaning
Mutual exclusion	A resource can be held by only one at a time
Hold and wait	A process holds one resource while waiting for another
No preemption	Resources can't be forcibly taken away
Circular wait	A cycle of processes each waiting on the next

P1 holds R1, wants R2
 P2 holds R2, wants R1

} circular wait
 → deadlock

FOUR STRATEGIES

Strategy	Idea
Prevention	Break one of the 4 conditions (e.g. force all locks acquired in a fixed global order → no circular wait)
Avoidance	Only grant a request if the system stays in a "safe state" (Banker's algorithm)
Detection	Let deadlocks happen, periodically find cycles in the wait-for graph
Recovery	Kill/rollback a victim process, or preempt its resources

BANKER'S ALGORITHM INTUITION

Like a cautious banker lending money: before granting a resource request, it simulates "if I grant this, is there still an order in which every process can finish with the resources left?" If yes (safe state), grant; if not, make the process wait. It avoids ever entering a state from which deadlock is unavoidable. Needs to know each process's maximum future needs in advance — often impractical, but a favorite exam topic.

MEMORY HOOK

Break any one of the four (Mutual exclusion, Hold-and-wait, No-preemption, Circular-wait) and deadlock is impossible. Easiest to break in practice: **circular wait**, by locking resources in a fixed order.

3.7 Memory Management INTERVIEW

WHY VIRTUAL MEMORY

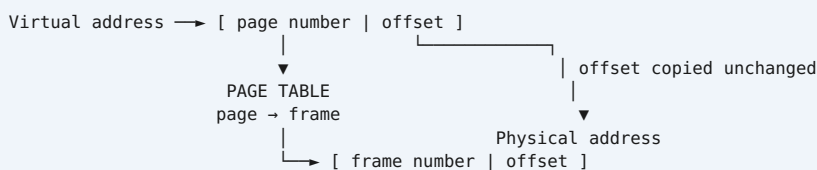
Two problems: (1) programs may need more memory than physically exists; (2) many processes must coexist without seeing or corrupting each other's memory. Virtual memory solves both: give each process its own huge **virtual address space**, and secretly map pieces of it to physical RAM (or disk) on demand.

ANALOGY

Every process gets its own private street-numbering (virtual addresses). The OS keeps a secret map (page table) translating "your house #5" to a real physical location, which may even be a storage unit across town (disk). Two processes can both have a "house #5" that map to different real places — that's isolation.

PAGING — THE CORE MECHANISM

Virtual and physical memory are both cut into fixed-size blocks: **pages** (virtual) and **frames** (physical), typically 4 KB. A **page table** maps each virtual page → a physical frame.



WORKED PAGING EXAMPLE

COMPUTE PAGE NUMBER & OFFSET

Page size = 4 KB = 4096 bytes = 2¹² → offset is the low 12 bits. For virtual address **8203**:

- page number = $8203 \div 4096 = 2$
- offset = $8203 \bmod 4096 = 8203 - 8192 = 11$

So virtual address 8203 = (page 2, offset 11). If page 2 maps to frame 7, physical address = $7 \times 4096 + 11 = 28683$.

- **TLB (Translation Lookaside Buffer)**: a small fast cache of recent page→frame translations, so the CPU usually skips the page-table lookup. A "TLB hit" is fast; a "TLB miss" falls back to the page table.
- **Page fault**: the requested page isn't in RAM. The OS traps, fetches it from disk into a frame, updates the page table, and resumes. This is expensive (disk is ~100,000× slower than RAM).
- **Demand paging**: load pages only when first accessed, not all upfront — faster start, less memory used.
- **Swapping**: moving whole pages/processes between RAM and disk (swap space) when RAM is tight.

SEGMENTATION VS PAGING

Paging = fixed-size blocks (no external fragmentation, but internal fragmentation in the last page). **Segmentation** = variable-size logical units (code, stack, heap segments) matching program structure, but suffers external fragmentation. Modern systems mostly use paging (sometimes both).

	Internal fragmentation	External fragmentation
What	Wasted space inside an allocated block (last page half-empty)	Free memory exists but in scattered small gaps, none big enough
Caused by	Fixed-size allocation (paging)	Variable-size allocation (segmentation)

THRASHING

THRASHING

When RAM is so overcommitted that processes spend more time paging in/out from disk than doing real work, throughput collapses. Sign: CPU is idle but disk is thrashing. Fixes: reduce multiprogramming degree, add RAM, use a better replacement policy. Concept to name: the **working set** — the pages a process actively needs; keep the working set in RAM to avoid thrashing.

PAGE REPLACEMENT (CHOOSE A VICTIM WHEN RAM IS FULL)

Reference string **7 0 1 2 0 3 0 4**, **3 frames**. Count page faults (f = fault, h = hit):

Ref:	7	0	1	2	0	3	0	4	
FIFO:	f	f	f	f	h	f	f	f	→ 7 faults (evicts oldest loaded)
LRU:	f	f	f	f	h	f	h	f	→ 6 faults (evicts least-recently used)
OPT:	f	f	f	f	h	f	h	f	→ 6 faults (evicts used-farthest-in-future)

Policy	Evicts	Note
FIFO	Oldest loaded page	Simple; can suffer Belady's anomaly (more frames → more faults!)
LRU	Least recently used	Good approximation of optimal; needs usage tracking
Optimal	The one used farthest in the future	Theoretical best (needs the future) — a benchmark, not implementable

QUIZ 3.D

Which replacement policy can, counter-intuitively, cause *more* page faults when you add more frames? What's that phenomenon called? (Answer at end.)

3.8 "What happens when you run a program?" INTERVIEW

The single most important OS interview question — it ties everything together.

1. SOURCE CODE (program.c)
| compiler + linker
- ↓
2. EXECUTABLE (binary on disk: code, data, symbols)
| you type ./program → shell calls fork() + exec()
- ↓
3. PROCESS CREATED — OS builds a PCB, assigns PID
- ↓
4. VIRTUAL ADDRESS SPACE set up — code/data/heap/stack laid out,
| pages NOT all loaded yet (demand paging)
- ↓
5. SCHEDULER puts process in READY queue → eventually RUNNING on CPU
- ↓
6. CPU executes instructions. First access to a page → PAGE FAULT →
| OS loads that page from disk into a RAM frame, updates page table
- ↓
7. Program needs I/O (read file, network) → SYSTEM CALL →
| trap into KERNEL MODE → kernel talks to HARDWARE via drivers
- ↓
8. Result returns → back to USER MODE → process continues
| on time-slice expiry → CONTEXT SWITCH to another process
- ↓
9. Program finishes → exit() → OS frees memory, closes files, removes PCB

CONNECTS TO

This journey touches every OS topic: process/PCB (3.2), scheduling (3.4), virtual memory + paging + page fault (3.7), system calls + user/kernel mode (3.1), and context switching. It also connects up the stack: step 7's I/O is often a **network** call (Section 4) or a **DB query** (Section 2). OS synchronization (mutex/semaphore) is the same idea as DB locks/transactions.

3.9 Confusion Pairs

Pair	Difference
Program vs Process	Passive file vs running instance with state
Process vs Thread	Isolated memory vs shared memory within a process
User vs Kernel mode	Restricted vs privileged CPU mode
Mutex vs Semaphore	Ownership lock (1) vs counter/signal (N)
Paging vs Segmentation	Fixed blocks vs variable logical units
Internal vs External fragmentation	Waste inside a block vs scattered free gaps
Page fault vs Segfault	Valid page not in RAM (recoverable) vs illegal memory access (crash)
Deadlock vs Starvation	Cyclic permanent wait vs indefinitely postponed but not cyclic
Preemptive vs Non-preemptive	OS can interrupt running process vs runs to completion/block

3.10 Chapter Interview Kit

RAPID RECALL

1. What does an OS actually manage? (name 3)
2. User mode vs kernel mode?
3. Program vs process?
4. What's in a PCB?
5. What is a context switch and why costly?
6. Process vs thread — one key difference?
7. Turnaround vs waiting time formulas?
8. Which scheduler minimizes average waiting time?
9. Race condition in one sentence?
10. Mutex vs semaphore?
11. Four deadlock conditions?
12. What is a page fault?
13. Paging vs segmentation?
14. What is thrashing?

INTERVIEW QUESTIONS

1. Walk me through what happens when I run `./a.out`.
2. How does virtual memory give isolation and the illusion of large memory?
3. Compute waiting/turnaround for these processes under FCFS vs SJF.
4. Explain a race condition and fix it with a mutex.
5. How would you prevent deadlock in code that takes multiple locks?
6. What's the TLB and why does it matter?
7. Difference between user-level and kernel-level threads.
8. What is demand paging and why is it good?
9. Explain LRU vs FIFO page replacement.
10. Why is a system call more expensive than a function call?

DEEP QUESTIONS

1. Why does too-frequent context switching reduce throughput even though each switch is fast? (Cache/TLB cold, pure overhead.)
2. SJF has optimal average waiting time — why isn't it used directly in real OSes? (Can't know future burst; starvation.)
3. Adding RAM sometimes doesn't fix slowness — when? (Not memory-bound; or Belady's anomaly with FIFO.)
4. Why can threads improve throughput on I/O-bound work even on one core? (Overlap I/O waits with compute.)
5. How are DB transactions and OS synchronization the same problem?

TRAP QUESTIONS

1. "More threads always = faster." (No — contention, context-switch overhead, GIL in some languages.)
2. "A page fault is an error/crash." (No — it's normal; a segfault is the crash.)
3. "RR gives the best average waiting time." (No — best responsiveness.)
4. "Semaphore and mutex are interchangeable." (No — ownership vs signaling.)
5. "Deadlock and starvation are the same." (No.)

EXPLAIN-IT-ALoud

- Explain the difference between a process and a thread to a junior developer.
- Explain virtual memory and paging with the page-table diagram.
- Explain a race condition and how a mutex fixes it, aloud.
- Explain "what happens when you run a program" end to end.

30-SECOND EXPLANATIONS

PROCESS VS THREAD IN 30S

A process is a running program with its own isolated memory; a thread is a lightweight execution path inside a process that shares the process's memory. Threads are cheaper to create and switch and can run in parallel, but because they share memory, concurrent access needs synchronization or you get race conditions.

VIRTUAL MEMORY IN 30S

Each process gets its own large virtual address space, and the OS maps its pages to physical RAM frames via a page table, loading pages from disk on demand. This gives isolation between processes and the illusion of more memory than exists; the TLB caches translations, and a page fault fetches a missing page from disk.

ONE PICTURE FOR THE WHOLE CHAPTER

The OS is a juggler standing between many programs and one set of hardware. It slices the CPU (scheduling), fakes unlimited private memory for each (virtual memory), lets programs safely request privileged actions (system calls), and coordinates their shared access (synchronization, deadlock handling) — all to keep the illusion that each program is alone on the machine.

Chapter 3 — Quiz Answers

ANSWERS

- 3.A** No. Two processes running the same program get **separate** address spaces — each has its own copy of the variables. Changing a variable in one doesn't affect the other. (Threads within one process *would* share.)
- 3.B** Starvation = a process waits indefinitely because others keep getting preferred. Classically caused by **Priority scheduling** (and SJF/SRTF for long jobs). Standard fix: **aging** — gradually raise the priority of long-waiting processes.
- 3.C** A **counting semaphore** initialized to **3**. Each thread does `wait()` to acquire a connection (decrement) and `signal()` to release (increment); the 4th concurrent thread blocks until one is freed. A mutex would wrongly limit it to 1.
- 3.D** **FIFO** can cause more page faults with more frames — this is **Belady's anomaly**. LRU and Optimal don't suffer from it (they're "stack" algorithms).

SECTION 4

Computer Networks

Networking is how two programs on different machines talk. The whole field is one idea repeated: **break a hard problem into layers**, where each layer does one job and trusts the layer below. Master the layers and everything — TCP, HTTP, DNS, TLS — slots into place.

4.1 The basics FOUNDATION

WHY NETWORKING

Data and users are spread across machines and the planet. Networking lets programs share data and services regardless of location. But the physical world is messy — wires fail, packets get lost, machines have different addresses — so we need agreed **protocols** (rules) so any two machines can understand each other.

Term	Meaning	Analogy
LAN	Local Area Network — one building/home	Rooms in one house
WAN	Wide Area Network — cities/countries (the Internet)	The postal system between cities
IP address	Logical address of a machine on a network (can change)	Your mailing address
MAC address	Hardware address burned into the network card (fixed)	Your fingerprint / device serial
Port	A number identifying a specific program/service on a machine	Apartment number at an address
Packet	A small chunk of data with headers, sent independently	One envelope in a multi-envelope letter

IP + PORT TOGETHER

IP address gets you to the building (the machine); the port gets you to the right apartment (the program). `142.250.x.x:443` = "Google's machine, the HTTPS service." Web = 80 (HTTP) / 443 (HTTPS), SSH = 22, DNS = 53.

4.2 The layered models INTERVIEW

WHY LAYERS

Sending "hello" across the world involves electrical signals, addressing, routing, reliability, and app formatting — too much for one blob of code. Layering splits it so each layer solves one thing and can change independently. HTTP doesn't care if you're on Wi-Fi or fiber; TCP doesn't care whether you're sending a webpage or an email.

OSI (7 layers, teaching model)	TCP/IP (4 layers, real world)	Example
7 Application	Application	HTTP, DNS
6 Presentation	(encoding, encryption)	TLS, JSON
5 Session		
4 Transport	Transport	TCP, UDP (ports)
3 Network	Internet	IP (routing)
2 Data Link	Link (Network Access)	Ethernet, MAC
1 Physical	(cables, Wi-Fi, signals)	

WHAT EACH CORE LAYER DOES

Layer	Job	Addresses by	Unit
Application	App-level meaning (web, mail, DNS)	—	Message
Transport	Program-to-program delivery, reliability	Port	Segment
Network	Machine-to-machine, routing across networks	IP	Packet
Link	Node-to-node on the same physical network	MAC	Frame

ENCAPSULATION: HOW DATA GOES DOWN THE STACK

```

Sending "GET /":
App:  [ HTTP: GET / ]
TCP:  [ TCP hdr | HTTP: GET / ]      ← adds ports, seq numbers
IP:   [ IP hdr | TCP hdr | HTTP ]    ← adds source/dest IP
Link: [ Eth hdr | IP hdr | TCP hdr | HTTP ] ← adds MAC addresses
      → out the wire →
Receiver peels each header off in reverse (de-encapsulation).

```

MEMORY HOOK (TOP TO BOTTOM)

"All People Seem To Need Data Processing" (Application, Presentation, Session, Transport, Network, Data-link, Physical). For the real world, remember just four: **App → Transport → Network → Link**.

4.3 Transport layer: TCP vs UDP INTERVIEW

	TCP	UDP
Connection	Connection-oriented (handshake first)	Connectionless (just send)
Reliability	Guaranteed, ordered, no duplicates	Best-effort; may lose/reorder
Speed/overhead	Slower, more overhead	Fast, lightweight
Flow/congestion control	Yes	No
Use for	Web, email, file transfer, anything that must arrive	Video/voice calls, gaming, DNS, live streams

ANALOGY

TCP is a registered letter with delivery confirmation and re-sending if lost — reliable but slower. UDP is shouting across a room — fast, but if a word is missed, it's gone. For a live call, a lost syllable isn't worth re-sending (it'd arrive late and useless) — that's why calls use UDP.

THE 3-WAY HANDSHAKE (ESTABLISHING A TCP CONNECTION)

```

Client                                Server
| — SYN (seq=x) —> |                  | "let's talk, my seq is x"
| <— SYN-ACK (seq=y, ack=x+1) — |      | "ok; my seq is y, got your x"
| — ACK (ack=y+1) —> |                  | "got yours; connected"
| ===== connection established ===== |

```

WHY 3 STEPS, NOT 2?

Both sides must confirm they can send **and** receive, and agree on starting sequence numbers. Two messages would only confirm one direction. The third ACK confirms the server's sequence number reached the client. (Termination is a **4-way** handshake: FIN/ACK each way, because each side closes its direction independently.)

RELIABILITY MECHANISMS

- **Sequence numbers + ACKs:** each byte is numbered; the receiver acknowledges what it got, so the sender knows what arrived and in what order.
- **Retransmission:** if an ACK doesn't come within a timeout, the sender resends.

- **Flow control:** the receiver advertises a *window* (how much buffer it has free) so a fast sender doesn't overwhelm a slow receiver.
- **Congestion control:** TCP probes the network capacity (slow start, then backs off on loss) so senders collectively don't overwhelm the *network*.

FLOW CONTROL VS CONGESTION CONTROL (CONFUSION)

Flow control protects the *receiver* (don't send faster than it can consume). **Congestion control** protects the *network* (don't send faster than the routers/links can carry). Different problems, both in TCP.

QUIZ 4.A

Why does DNS typically use UDP but a large DNS response or zone transfer may fall back to TCP? (Answer at end.)

4.4 HTTP FOUNDATION INTERVIEW

WHAT HTTP IS

HyperText Transfer Protocol — the application-layer request/response format the web speaks over TCP. A client sends a request; the server sends a response. It is **stateless**: each request is independent, the server remembers nothing between them by default (that's what cookies/sessions fix).

<pre>REQUEST GET /users/42 HTTP/1.1 Host: api.example.com Authorization: Bearer abc Accept: application/json (blank line) (body, if any)</pre>	↔	<pre>RESPONSE HTTP/1.1 200 OK Content-Type: application/json Cache-Control: max-age=60 (blank line) { "id": 42, "name": "Asha" }</pre>
<pre>method path version headers body</pre>		<pre>version status reason headers body</pre>

METHODS (VERBS)

Method	Purpose	Safe?	Idempotent?
GET	Read a resource	Yes	Yes
POST	Create / trigger action	No	No
PUT	Replace a resource	No	Yes
PATCH	Partially update	No	Not necessarily
DELETE	Remove a resource	No	Yes

Idempotent = doing it N times has the same effect as once (important for safe retries). GET/PUT/DELETE are; POST isn't.

STATUS CODES (KNOW THE FAMILIES)

Range	Meaning	Examples
1xx	Informational	100 Continue
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirect	301 Moved Permanently, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Unavailable

TRAP: 401 VS 403

401 Unauthorized = "I don't know who you are" (not authenticated — log in). **403 Forbidden** = "I know who you are, but you're not allowed" (authenticated but lacks permission). This maps exactly to Authentication vs Authorization (Section 5).

COOKIES, SESSIONS, CACHING

- **Cookie**: a small piece of data the server sets (`Set-Cookie`) that the browser stores and sends back on each request — used to keep state despite HTTP being stateless.
- **Session**: server-side state (e.g. logged-in user), identified by a session ID stored in a cookie.
- **Caching**: `Cache-Control` , `ETag` , and `304 Not Modified` let clients/proxies reuse responses instead of re-fetching, saving bandwidth and latency.

REST APIS

REST = a style for web APIs: model everything as **resources** with URLs, use HTTP methods as verbs, and keep it stateless. `GET /users/42` reads user 42; `POST /users` creates one. (More in Section 5.)

4.5 HTTPS & TLS INTERVIEW

WHY HTTPS

Plain HTTP travels in the clear — anyone on the path (café Wi-Fi, ISP) can read or tamper with it. HTTPS = HTTP over **TLS**, which adds three things: **encryption** (eavesdroppers see gibberish), **integrity** (tampering is detected), and **authentication** (you're really talking to the real server, not an impostor).

THE TWO KINDS OF ENCRYPTION USED

- **Asymmetric** (public/private key): slow, but lets strangers exchange a secret securely. The server has a public key (shared) and private key (secret).
- **Symmetric** (one shared key): fast; used for the actual data once a shared key is established.

TLS HANDSHAKE INTUITION

(1) Client says hello, lists supported ciphers. (2) Server sends its **certificate** containing its public key, signed by a trusted **Certificate Authority (CA)**. (3) Client verifies the certificate chains up to a CA it trusts (proving the server's identity) and checks it matches the domain. (4) They use asymmetric crypto to agree on a shared **symmetric** session key. (5) All further traffic is encrypted with that fast symmetric key. So: asymmetric to *establish trust + a key*, symmetric to *do the bulk work*.

```

Client                                     Server
| - ClientHello (ciphers) -> |
| <- ServerHello + CERTIFICATE (public key) - |
| verify cert via trusted CA ✓ |
| agree on shared symmetric key (key exchange) |
| == encrypted with symmetric session key == |

```

TRAP

"HTTPS encrypts everything including the domain and URL." The **path, headers, and body are encrypted**, but the destination IP and the domain name (via SNI, and via the DNS lookup) are typically visible to the network. So HTTPS hides *what* you send, not entirely *where* you're going.

QUIZ 4.B

Why not just use fast symmetric encryption for everything and skip the slow asymmetric part? (Answer at end.)

4.6 DNS & "What happens when you type google.com" INTERVIEW

WHY DNS

Humans remember names (`google.com`); machines route by IP (`142.250.x.x`). The **Domain Name System** is the Internet's phone book, translating names to IPs. It's distributed and cached at every level so it scales to the whole planet.

QUIZ 4.C

A user in Australia loads a site hosted in the US and images appear instantly while the page HTML is a bit slower. What's likely serving the images? (Answer at end.)

4.8 Confusion Pairs

Pair	Difference
TCP vs UDP	Reliable + ordered + connection vs fast + best-effort
HTTP vs HTTPS	Plaintext vs encrypted (TLS) + authenticated
IP vs MAC address	Logical, changeable, routable vs physical, fixed, local
Flow vs Congestion control	Protect receiver vs protect network
401 vs 403	Not authenticated vs authenticated-but-forbidden
Proxy vs Reverse proxy	Fronts clients vs fronts servers
Symmetric vs Asymmetric encryption	One shared key (fast) vs key pair (secure exchange)
OSI vs TCP/IP model	7-layer teaching model vs 4-layer real model
Stateless vs Stateful	Each request independent (HTTP) vs server remembers context

4.9 Chapter Interview Kit

RAPID RECALL

1. IP vs MAC vs port?
2. Four TCP/IP layers?
3. TCP vs UDP — one deciding factor?
4. Steps of the 3-way handshake?
5. Why 3 steps not 2?
6. Flow vs congestion control?
7. Name the 5 status-code families.
8. 401 vs 403?
9. What's a cookie for?
10. What does TLS add to HTTP?
11. Asymmetric vs symmetric — who does what in TLS?
12. What does DNS do?
13. Proxy vs reverse proxy?
14. When WebSockets over HTTP polling?

INTERVIEW QUESTIONS

1. Walk me through exactly what happens when I type google.com.
2. Explain the TCP 3-way handshake and why each step exists.
3. How does HTTPS keep data private and prove server identity?
4. Why is HTTP called stateless, and how do we build logged-in sessions on top of it?
5. TCP vs UDP — give a real use case for each.
6. What's the difference between a load balancer and a CDN?
7. Explain the OSI vs TCP/IP models.
8. What happens during DNS resolution, and where is caching involved?
9. Explain idempotency and why it matters for retries.
10. How does a reverse proxy help with security and scaling?

DEEP QUESTIONS

1. If TLS gives encryption, what does the certificate/CA add on top? (Authentication — without it you'd encrypt to an impostor.)
2. Why does congestion control benefit everyone even though it slows an individual sender? (Prevents collapse; shared resource.)
3. Why is UDP better for a video call despite being "unreliable"? (Late data is useless; drop > delay.)
4. HTTP is stateless but real apps feel stateful — where does the state actually live? (Cookies + server sessions/DB/tokens.)
5. Why can NAT complicate peer-to-peer connections? (Devices lack public reachable addresses.)

TRAP QUESTIONS

1. "HTTPS hides the website you're visiting completely." (No — IP/domain often visible.)
2. "TCP guarantees data arrives instantly." (No — reliable, not fast; adds latency.)
3. "A MAC address changes when you move networks." (No — IP changes; MAC is fixed.)
4. "POST is idempotent." (No.)
5. "DNS uses TCP always." (Usually UDP; TCP for large responses.)

EXPLAIN-IT-ALoud

- Explain TCP vs UDP to a junior dev with an analogy.
- Explain the 3-way handshake, drawing SYN/SYN-ACK/ACK.
- Explain what happens when you type a URL, end to end.
- Explain how HTTPS proves you're talking to the real server.

30-SECOND EXPLANATIONS

TCP VS UDP IN 30S

TCP is connection-oriented and reliable: it does a handshake, numbers bytes, acknowledges them, retransmits losses, and controls flow and congestion — ideal when every byte must arrive (web, files). UDP is connectionless and best-effort: no handshake, no guarantees, low overhead — ideal when speed beats completeness (calls, gaming, DNS).

HTTPS IN 30S

HTTPS is HTTP over TLS. In the handshake the server presents a CA-signed certificate proving its identity; the client verifies it and they use asymmetric crypto to agree on a fast symmetric session key. All subsequent traffic is encrypted with that key, giving confidentiality, integrity, and authentication.

ONE PICTURE FOR THE WHOLE CHAPTER

Networking is nested envelopes. Your message (HTTP) goes inside a reliability envelope (TCP), inside a routing envelope (IP), inside a physical-delivery envelope (Ethernet/Wi-Fi). Each machine along the way reads only the envelope it's responsible for. DNS finds the address, TLS locks the envelope, and the layers let each part evolve independently.

Chapter 4 — Quiz Answers

ANSWERS

4.A DNS uses UDP because a lookup is one small request/response — the handshake overhead of TCP isn't worth it, and a lost query is just retried. But if the response is too large to fit in a single UDP packet (or for zone transfers), it falls back to TCP, which handles large, reliable, ordered transfers.

4.B Because two strangers can't share a symmetric key over an insecure network without an eavesdropper capturing it. Asymmetric crypto solves the *key-exchange* problem: it lets them agree on a symmetric key securely. Once agreed, they switch to fast symmetric encryption for the bulk data. Asymmetric = establish trust + key; symmetric = do the work.

4.C A **CDN** — the static images are cached on edge servers near Australia, so they load fast, while the dynamic HTML still comes from the origin server in the US (farther away).

SECTION 5

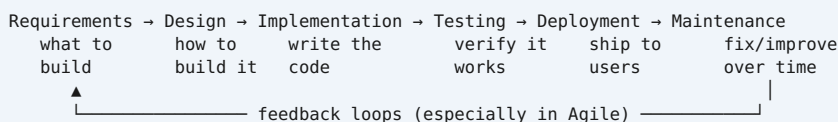
Software Engineering & SDE Fundamentals

This is the "how professionals actually build and ship software" chapter. It's less about a single algorithm and more about the vocabulary and mental models interviewers use to check you've worked on real systems: process, design quality, testing, APIs, backend architecture, and light system design.

5.1 The Software Development Life Cycle (SDLC) FOUNDATION

WHY A PROCESS AT ALL

Left to chance, software projects drift: unclear requirements, surprises late, untested code shipped. The SDLC is a repeatable sequence of stages so teams know *what to do when* and catch problems early — because a bug found in requirements costs pennies; the same bug found in production costs a fortune.



Phase	Output
Requirements	What the system must do (functional) + qualities like speed/security (non-functional)
Design	Architecture, components, data model, APIs
Implementation	The code
Testing	Verification that it meets requirements
Deployment	Release to users
Maintenance	Bug fixes, improvements, scaling

5.2 Methodologies INTERVIEW

	Waterfall	Agile
Shape	Sequential; finish each phase before the next	Iterative; small increments, frequent feedback
Change	Hard/expensive late	Embraced continuously
Delivery	One big release at the end	Working software every iteration (sprint)
Best for	Fixed, well-understood requirements (e.g. regulated hardware)	Evolving requirements (most modern software)
Risk	Discover problems only at the end	Discover early, adjust

SCRUM VS KANBAN (BOTH AGILE)

Scrum: work in fixed-length **sprints** (e.g. 2 weeks) with defined roles (Product Owner, Scrum Master, Dev Team) and ceremonies (planning, daily standup, review, retrospective). **Kanban:** continuous flow, no fixed sprints; visualize work on a board and limit work-in-progress (WIP) to expose bottlenecks. Scrum = timeboxed batches; Kanban = a steady stream.

Kanban board (limit WIP per column):



WIP limits stop everyone starting new work and finishing nothing.

5.3 Software design principles INTERVIEW

All of these serve one goal (same as OOP's): **make code easy to change without breaking things.**

Principle	Meaning	Why
Modularity	Split system into independent parts	Understand/change one piece at a time
Abstraction	Expose what, hide how	Use components without knowing internals
Cohesion (want HIGH)	How focused a module is on one job	Focused modules are easier to reason about
Coupling (want LOW)	How dependent modules are on each other	Loose coupling → change one without touching others
Separation of concerns	Different responsibilities in different places	UI, logic, data don't tangle
DRY	Don't Repeat Yourself — one source of truth	Fix a rule in one place
KISS	Keep It Simple	Simple code has fewer bugs
YAGNI	You Aren't Gonna Need It — don't build for imagined futures	Avoid wasted, complex speculation

MEMORY HOOK

Aim for **high cohesion, low coupling**. A good module is like a good employee: deeply focused on its own job (cohesion) and not tangled in everyone else's business (coupling).

TRAP: DRY TAKEN TOO FAR

Over-DRYing couples unrelated things that just *happened* to look similar. If two pieces of code are identical by coincidence, not by shared meaning, forcing them together creates fragile coupling. "Duplication is cheaper than the wrong abstraction."

SOLID (from Section 1.7) is the OOP-specific set of these principles — reuse that knowledge here.

5.4 Testing INTERVIEW

WHY THE PYRAMID

Tests catch regressions so you can change code fearlessly. But not all tests are equal: fast, isolated tests give quick feedback; broad end-to-end tests are slow and flaky. So you want **many small tests, few big ones** — the test pyramid.



E2E / Acceptance (few, slow, real user flows)

Integration (some, components together)

Unit (many, fast, one function/class)

Level	Tests	Scope
Unit	One function/class in isolation	Smallest; fast; dependencies faked
Integration	Multiple components together (e.g. service + real DB)	Interactions, wiring
System	The whole system end-to-end	Full stack behavior
Acceptance	Does it meet the business requirement?	User/stakeholder perspective
Regression	Re-run old tests after changes	Ensure nothing old broke

MOCK VS STUB (CLASSIC CONFUSION)

Both are test doubles that replace real dependencies. A **stub** provides canned answers ("when asked for user 5, return this fake user") — you check the *output* of the code under test. A **mock** also records and *verifies interactions* ("assert that `emailService.send()` was called exactly once") — you check *behavior*. Stub = state verification; mock = behavior verification.

```
// Unit test with a stub: replace the DB so the test is fast + isolated
paymentRepo = stub();           // returns a fake balance
stub.getBalance("A") returns 1000;
result = accountService.withdraw("A", 300); // real logic under test
assert result.balance == 700;               // verify OUTPUT (stub)

// Mock: verify an interaction happened
emailService = mock();
accountService.withdraw("A", 300);
verify(emailService).send(once);           // verify BEHAVIOR (mock)
```

QUIZ 5.A

Your test spins up a real Postgres database and checks that saving then loading a user works. Unit or integration test? (Answer at end.)

5.5 APIs, Auth, and the web contract INTERVIEW

An **API (Application Programming Interface)** is a contract: how one piece of software asks another to do something. A **REST API** exposes resources over HTTP (recap Section 4.4): URLs are nouns, HTTP methods are verbs, data is usually **JSON**, and it's stateless.

```
GET    /users    → list users
POST   /users    → create a user    (body: JSON)
GET    /users/42 → read user 42
PUT    /users/42 → replace user 42
DELETE /users/42 → delete user 42
```

AUTHENTICATION VS AUTHORIZATION (KNOW COLD)

AUTHN VS AUTHZ

Authentication = *who are you?* (verify identity — password, token). **Authorization** = *what are you allowed to do?* (permissions/roles). You authenticate first, then authorize. Maps to HTTP **401** (not authenticated) vs **403** (authenticated but forbidden).

SESSIONS VS TOKENS (JWT)

	Session (cookie)	Token (JWT)
State	Server stores session; cookie holds an ID	Stateless; token carries the claims, signed
Scale	Server must share session store	Any server can verify the signature
Revocation	Easy (delete session)	Harder (valid until expiry)

HOW A JWT WORKS

A **JSON Web Token** has three parts: **header.payload.signature**. The payload holds claims (user id, roles, expiry). The server signs it with a secret; anyone can read the payload (it's just base64, **not encrypted**) but only the server can produce a valid **signature**. On each request the client sends the token; the server verifies the signature to trust the claims — no server-side session lookup needed.

TRAP

"JWTs are encrypted, so it's safe to put secrets in them." No — a standard JWT is **signed, not encrypted**. Anyone can decode and read the payload. Never put passwords or sensitive data in it.

5.6 Backend architecture & request lifecycle INTERVIEW

WHY LAYERED BACKEND

Cramming routing, business rules, and SQL into one function makes it untestable and unchangeable. Splitting into layers (separation of concerns) means you can swap the DB, test business logic in isolation, and reuse services — the same "make change safe" goal again.

Request lifecycle through a layered backend:



Layer	Responsibility	Should NOT
Controller	HTTP in/out, validation, status codes	Contain business rules or SQL
Service	Business logic, orchestrate, transactions	Know about HTTP or SQL syntax
Repository	Read/write data, hide DB details	Contain business rules

TRAP: THE N+1 QUERY PROBLEM

Fetch 100 orders, then loop and query each order's customer separately → $1 + 100 = 101$ queries. Fix with a **join** or eager loading (1 query). This lives at the repository layer and is a very common interview + real-world performance bug (connects to DB indexing, Section 2).

THE SUPPORTING CAST (OPERATIONAL CONCERNS)

Concept	What / why
Caching	Store hot results (in memory / Redis) to avoid recomputing or re-querying. Huge latency win; risk = stale data (cache invalidation).
Message queue	A buffer (Kafka, RabbitMQ, SQS) between producers and consumers — decouples services and smooths spikes.
Async / background jobs	Do slow work (send email, generate report) outside the request so the user gets a fast response.
Logging	Record what happened (structured logs) — essential for debugging production.
Monitoring / metrics	Track health (latency, errors, CPU); alert when things break.
Error handling	Fail gracefully; return meaningful errors; don't leak internals.
Config & env vars	Keep settings/secrets out of code; inject per environment (dev/staging/prod). "12-factor" style.

MESSAGE QUEUE ANALOGY

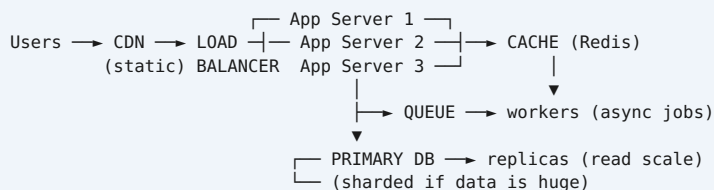
A restaurant. Waiters (producers) drop orders on a rail (queue); cooks (consumers) pull them at their own pace. If a rush hits, orders queue up instead of overwhelming the kitchen, and waiters aren't blocked waiting for each dish.

QUIZ 5.B

A user uploads a video that takes 2 minutes to process. Should the HTTP request wait for processing to finish? What pattern fixes this? (Answer at end.)

5.7 Light system design INTERVIEW DEEPER

Enough to reason about "how would this scale?" — not a full course.



SCALING: VERTICAL VS HORIZONTAL

	Vertical (scale up)	Horizontal (scale out)
How	Bigger machine (more CPU/RAM)	More machines
Limit	Hardware ceiling; single point of failure	Nearly unlimited; needs load balancing
Complexity	Simple	Higher (stateless services, coordination)

MEMORY HOOK

Vertical = a stronger horse. Horizontal = more horses. Beyond a point, you can't buy a bigger horse — you add horses, which is why scalable systems are built **stateless** so any server can handle any request.

Building block	Solves
Load balancer	Spread traffic; remove single point of failure
Cache	Reduce repeated expensive work / DB load
Replication	Scale reads + high availability
Sharding	Scale data beyond one machine
Queue	Decouple services; absorb spikes; async work
CDN	Serve static content near users

CONNECTS TO

Everything converges here. The layered backend runs as **processes/threads** (OS, §3), handling **HTTP over TCP/TLS** (Networks, §4), running **OOP-designed code** (§1), talking to a **database with transactions and indexes** (§2), deployed in **containers** (§7), versioned with **Git** (§6). Load balancing, replication, sharding, and CDNs are the DB/network concepts scaled up.

5.8 Confusion Pairs

Pair	Difference
Cohesion vs Coupling	Focus within a module (want high) vs dependence between modules (want low)
Waterfall vs Agile	Sequential big-bang vs iterative feedback
Scrum vs Kanban	Timeboxed sprints vs continuous flow with WIP limits
Mock vs Stub	Verify behavior vs provide canned data
Unit vs Integration test	One isolated unit vs components together
Authentication vs Authorization	Who you are vs what you can do
Session vs JWT	Server-stored state vs stateless signed token
Horizontal vs Vertical scaling	More machines vs a bigger machine
Sync vs Async	Wait for result vs continue, handle later

5.9 Chapter Interview Kit

RAPID RECALL

1. Six SDLC phases?
2. Waterfall vs Agile?
3. Scrum vs Kanban?
4. High or low cohesion? Coupling?
5. DRY, KISS, YAGNI in one line each?
6. Levels of the test pyramid?
7. Mock vs stub?
8. Authentication vs authorization?
9. What are the backend layers?
10. What is the N+1 query problem?
11. Why use a message queue?
12. Horizontal vs vertical scaling?

INTERVIEW QUESTIONS

1. Walk a request through your backend layers and what each does.
2. How do you decide unit vs integration vs e2e coverage?

3. Explain authentication vs authorization and how JWT fits.
4. How would you scale a read-heavy service?
5. What is caching and what's the hardest part of it? (Invalidation.)
6. When would you make work asynchronous?
7. Explain high cohesion / low coupling with an example.
8. How do you keep secrets/config out of code?
9. What is idempotency and where does it matter in APIs?
10. Describe a message queue and a use case.

DEEP QUESTIONS

1. Why can over-applying DRY make code worse? (Wrong abstraction couples unrelated things.)
2. Why build stateless services for horizontal scaling? (Any server handles any request; no sticky state.)
3. JWT is stateless — what problem does that create and how do you mitigate? (Revocation; short expiry + refresh tokens.)
4. Caching improves latency but introduces which classic hard problem? (Invalidation/staleness.)
5. Why put business logic in services, not controllers or repositories? (Reuse + testability + separation.)

TRAP QUESTIONS

1. "JWTs are encrypted." (No — signed; payload is readable.)
2. "More tests = better." (No — slow/flaky e2e overuse hurts; balance the pyramid.)
3. "Agile means no planning/docs." (No.)
4. "Vertical scaling is unlimited." (No — hardware ceiling + SPOF.)
5. "Authentication and authorization are the same." (No.)

EXPLAIN-IT-ALoud

- Explain cohesion and coupling to a junior dev.
- Explain the journey of an HTTP request through the backend layers.
- Explain the difference between authentication and authorization with the 401/403 mapping.
- Explain when and why you'd introduce a message queue.

30-SECOND EXPLANATIONS

BACKEND LAYERS IN 30S

A request hits routing, passes through middleware (auth, logging), reaches a controller that handles HTTP concerns, which calls a service holding the business logic, which uses a repository to read/write the database. Each layer has one responsibility, so you can test logic in isolation and swap the DB without rewriting everything.

SCALING IN 30S

Vertical scaling means a bigger machine — simple but capped and a single point of failure. Horizontal scaling means more machines behind a load balancer — nearly unlimited but requires stateless services, plus caching, read replicas, sharding, and queues to scale the data and background work.

ONE PICTURE FOR THE WHOLE CHAPTER

Professional software is layered on purpose: a process to decide what to build, principles to keep code changeable, tests to change it safely, APIs as clean contracts between parts, a layered backend so each concern is isolated, and scaling patterns (LB, cache, replicas, queues) to grow. Every choice serves the same master: safe, sustainable change.

Chapter 5 — Quiz Answers

ANSWERS

5.A Integration test — it involves multiple real components (your code + a real database) working together, not one isolated unit. A pure unit test would fake/stub the database.

5.B No, the HTTP request should not block for 2 minutes (bad UX, ties up resources, likely times out). Fix: accept the upload, return immediately (e.g. 202 Accepted), and push a job onto a **message queue** for a **background worker** to process asynchronously; notify the user when done.

SECTION 6

Git — Version Control

Git is a time machine for code plus a merge engine for teamwork. Once you understand its **three areas** and that a commit is just a **snapshot with a parent pointer**, every command stops being magic incantation and becomes obvious.

6.1 The problem Git solves FOUNDATION

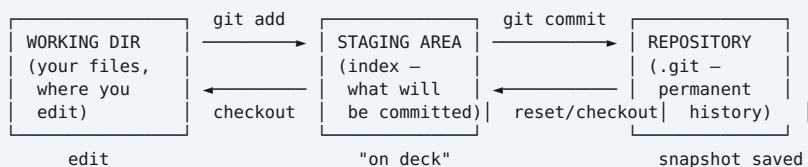
WHY VERSION CONTROL

Without it: `report_final.doc`, `report_final_v2.doc`, `report_final_REALLY.doc`. No history, no way to see what changed or undo safely, and total chaos when two people edit the same file. Version control records every change with who/when/why, lets you jump to any past state, and merges concurrent work. Git specifically is **distributed**: every clone has the full history, so you can work offline and there's no single point of failure.

ANALOGY

Git is a save-game system for your project. Every commit is a save point you can return to. Branches are parallel save files where you try things without risking the main game. Merge combines two save files' progress.

6.2 The three areas (the core mental model) FOUNDATION



WHY A STAGING AREA AT ALL?

It lets you craft a commit deliberately — choose exactly which changes go in, even parts of a file — instead of dumping everything. You might fix a bug and add a feature in your working directory but commit them **separately** for a clean history. Staging is the "shopping cart" before you "check out" (commit).

Area	What it is	Command to move in
Working directory	Actual files you edit	(edit files)
Staging area (index)	Changes marked for the next commit	<code>git add</code>
Repository (.git)	Committed, permanent snapshots	<code>git commit</code>

6.3 Everyday commands FOUNDATION

Command	Does
<code>git init</code>	Create a new repo (a <code>.git</code> folder) here
<code>git clone URL</code>	Copy a remote repo (full history) locally
<code>git status</code>	What's modified / staged / untracked
<code>git add file</code>	Stage changes for the next commit
<code>git commit -m "msg"</code>	Save staged changes as a snapshot
<code>git log</code>	View commit history
<code>git diff</code>	Show changes (working vs staged vs committed)
<code>git push</code>	Upload local commits to the remote
<code>git pull</code>	Fetch remote commits and merge them in
<code>git fetch</code>	Download remote commits but don't merge

FETCH VS PULL (VERY COMMON QUESTION)

`git fetch` downloads new commits from the remote into your local tracking branches but **leaves your working branch untouched** — you can inspect before integrating. `git pull` = `git fetch` + `git merge` (auto-integrates). Rule: `fetch` = look before you leap; `pull` = look and leap in one step.

6.4 Branches INTERVIEW

WHY BRANCHES

To work on a feature or fix without destabilizing the main line, and to let many people work in parallel. A branch is **cheap**: it's just a movable pointer to a commit, not a copy of files. You branch off, experiment freely, then merge back when ready.

Commit graph (each node = a snapshot, arrows point to PARENT):

```

      C4 — C5      ← feature branch
      /
A — B — C3      ← main
main and feature share history up to C3.
"HEAD" = pointer to where you currently are (the checked-out commit/branch).
```

Command	Does
<code>git branch feature</code>	Create a branch
<code>git switch feature</code> (or <code>checkout</code>)	Move onto it (HEAD points there)
<code>git merge feature</code>	Combine feature's commits into current branch
<code>git rebase main</code>	Re-apply your commits on top of main's latest

MERGE VS REBASE (THE FLAGSHIP GIT INTERVIEW QUESTION)

Before:

```

main:  A — B — C
feature:      |
              D — E
```

MERGE (main into feature or feature into main):

```

A — B — C ————— M      ← M = merge commit with TWO parents
                \         history preserved, but a "diamond"
                 D — E —
```

REBASE (feature onto main):

```

A — B — C — D' — E'      ← D,E replayed as NEW commits on top of C
                        (linear history, no merge commit; original D,E rewritten)
```


	Merge	Rebase
History	Preserves exactly what happened (with merge commit)	Rewrites into a clean linear line
Result	Non-destructive, true record	New commit hashes (history rewritten)
Good for	Integrating shared/public branches	Tidying your <i>local</i> branch before sharing
Golden rule	—	Never rebase commits others have already pulled

THE GOLDEN RULE OF REBASE

Never rebase **public/shared** history. Rebasing rewrites commits (new hashes); if teammates already based work on the old commits, you create duplicate/divergent history and painful conflicts. Rebase your own *local*, *unpushed* work to keep it tidy; merge when integrating shared branches.

MERGE CONFLICTS

A conflict happens when two branches changed the **same lines** differently — Git can't decide, so it marks the spot and asks you:

```

<<<<<< HEAD
color = "blue";      // your version
=====
color = "green";     // incoming version
>>>>>> feature
// You edit to the desired result, remove the markers, then: git add + commit

```

QUIZ 6.A

You made 3 messy local commits fixing typos in one feature and haven't pushed. Merge or rebase (interactive) to clean them up before opening a PR? (Answer at end.)

6.5 The rest you must know INTERVIEW

Term / command	Meaning
remote	A named reference to another copy of the repo (usually on a server)
origin	The default name for the remote you cloned from
HEAD	Pointer to your current commit/branch ("you are here")
<code>git reset</code>	Move the branch pointer back; can unstage or discard commits (rewrites local history)
<code>git revert</code>	Create a new commit that undoes a previous one (safe for shared history)
<code>git stash</code>	Temporarily shelve uncommitted changes to switch context, then reapply
<code>git cherry-pick</code>	Apply one specific commit from another branch onto this one
<code>.gitignore</code>	Lists files/patterns Git should not track (build output, secrets, <code>node_modules</code>)
tag	A named marker for a specific commit, usually a release (<code>v1.0.0</code>)

RESET VS REVERT (SAFE UNDO)

`git revert` is the **safe** undo for commits already pushed/shared: it adds a new commit that reverses the change, so history stays intact. `git reset` *rewrites* history (moves the pointer, drops commits) — fine for local unpushed work, dangerous on shared branches. "Revert forward, reset backward."

RESET MODES

`reset --soft` moves the branch pointer but keeps changes staged. `--mixed` (default) unstages them (keeps in working dir). `--hard` discards them entirely (destructive). Interview-ready: soft keeps everything staged, mixed keeps in working dir, hard throws away.

6.6 What happens internally when you commit INTERVIEW DEEPER

HOW A COMMIT REALLY WORKS

Git stores content as **objects** identified by a hash (SHA) of their content. On commit, Git: (1) snapshots the staged files as **blob** objects (file contents), (2) builds **tree** objects (directory structure pointing to blobs), (3) creates a **commit** object storing the top tree, the author/message, and a pointer to the **parent** commit, (4) moves the current branch pointer to the new commit. So history is a chain of snapshots linked by parent pointers — not a list of diffs. (Diffs are computed on demand by comparing snapshots.)

```
commit → tree → blob (file A)
      |   ↘   ↗ blob (file B)
      |   parent
      ↓
parent commit → tree → ...
```

Each commit = full snapshot (via shared blobs) + parent link. Immutable + hashed.

GIT VS GITHUB (ASKED CONSTANTLY)

Git is the version-control tool that runs on your machine (created by Linus Torvalds). **GitHub** (also GitLab, Bitbucket) is a *hosting service* for Git repositories, adding a web UI, pull requests, issues, CI/CD, and collaboration. Git works fully offline without GitHub; GitHub is optional convenience on top. "Git is the engine; GitHub is a garage that stores and shows off cars."

QUIZ 6.B

You committed a password to a shared branch and pushed it. Do you use `reset` or `revert` to stop it appearing going forward — and why is neither fully enough? (Answer at end.)

6.7 A realistic workflow

```
git clone URL           # get the repo
git switch -c feature/login # branch off
# ...edit files...
git status              # see what changed
git add .               # stage
git commit -m "Add login" # snapshot
git fetch origin         # see remote updates (safe)
git rebase origin/main   # replay my work on latest main (local, unpushed)
git push -u origin feature/login # share it
# open Pull Request → review → merge into main on GitHub
```

6.8 Confusion Pairs

Pair	Difference
Git vs GitHub	The tool (local) vs a hosting service (cloud)
fetch vs pull	Download only vs download + merge
merge vs rebase	Preserve history (merge commit) vs rewrite into linear
reset vs revert	Rewrite history (local) vs new undo commit (safe/shared)
Working dir vs Staging vs Repo	Edit vs on-deck vs saved history
HEAD vs branch	Where you are now vs a named line of commits
origin vs upstream	Your default remote vs (often) the original you forked from
tag vs branch	Fixed marker (release) vs moving pointer (development)

6.9 Chapter Interview Kit

RAPID RECALL

1. What problem does version control solve?
2. Name the three areas.
3. Why does the staging area exist?
4. fetch vs pull?
5. What is a branch, physically?
6. merge vs rebase in one line?
7. The golden rule of rebase?
8. reset vs revert?
9. Git vs GitHub?
10. What does HEAD point to?
11. What is `.gitignore` for?
12. What is a tag?

INTERVIEW QUESTIONS

1. Explain what happens internally when you commit.
2. When would you use rebase instead of merge?
3. How do you undo a commit safely on a shared branch?
4. Walk through resolving a merge conflict.
5. What are the three reset modes?
6. How would you apply just one commit from another branch? (cherry-pick)
7. How do you temporarily save work to switch branches? (stash)
8. Why is Git called distributed?
9. Difference between origin and a remote in general.
10. How does Git store history — diffs or snapshots?

DEEP QUESTIONS

1. Why does rebasing create new commit hashes, and why does that matter for shared branches?
2. Git stores snapshots, not diffs — how does it stay space-efficient? (Shared blobs by hash + packfiles.)
3. Why is a commit hash based on content and parent? (Integrity + immutability + dedup.)
4. Why can two people commit at the same time without a central lock? (Distributed; merge later.)
5. When is `reset --hard` the right tool and when is it a footgun?

TRAP QUESTIONS

1. "Git and GitHub are the same." (No.)
2. "Rebase is always better than merge." (No — not on shared history.)
3. "Git stores diffs between versions." (No — snapshots linked by parents.)
4. "`git pull` only downloads." (No — it also merges.)
5. "reset is a safe undo for pushed commits." (No — use revert.)

EXPLAIN-IT-ALoud

- Explain the three Git areas to a junior dev.
- Explain merge vs rebase with a commit graph.
- Explain what a commit actually is internally.
- Explain why you'd use revert over reset on `main`.

30-SECOND EXPLANATIONS

GIT IN 30S

Git is a distributed version-control system. You edit in the working directory, stage the changes you want with `git add`, and snapshot them with `git commit` into the repository. Each commit is an immutable, hash-identified snapshot pointing to its parent, forming a history you can branch, merge, and rewind — and every clone holds the whole history.

MERGE VS REBASE IN 30S

Merge combines two branches and creates a merge commit, preserving the true, possibly-branchy history. Rebase replays your commits on top of another branch as new commits, producing a clean linear history but rewriting hashes — so rebase local work to tidy it, and never rebase commits others already share.

ONE PICTURE FOR THE WHOLE CHAPTER

Three boxes and a chain of snapshots: edit in the working directory, stage into the index, commit into the repository as an immutable snapshot linked to its parent. Branches are cheap pointers into that chain; merge and rebase are two ways to bring lines of history back together.

Chapter 6 — Quiz Answers

ANSWERS

6.A Interactive rebase (`git rebase -i`) to squash the messy local commits into one clean commit — it's safe because the work is **local and unpushed**, so you're not rewriting shared history. This gives a tidy PR.

6.B Use `git revert` to add a commit removing the password going forward (safe on a shared branch; `reset` would rewrite pushed history and disrupt teammates). But neither is fully enough: the secret still exists in **history** and in others' clones, so you must also **rotate/invalidate the leaked credential** and, if truly needed, purge history with tools like `git filter-repo` and force-coordinate with the team.

SECTION 7

Docker — Containers

Docker packages an application **with everything it needs to run** into a portable box that behaves identically on your laptop, a teammate's machine, and production. If you remember one thing: **an image is the recipe, a container is the running dish.**

7.1 The problem Docker solves FOUNDATION

"IT WORKS ON MY MACHINE"

Your app depends on a specific runtime version, libraries, OS packages, and config. On another machine any of those differs, and it breaks. Docker's fix: bundle the app **and all its dependencies and environment** into one image, so the environment travels with the code. The machine's own setup no longer matters — if it runs Docker, it runs your image identically.

Without Docker: App needs Node 18, libX 2.1 Dev machine: Node 20 x Prod: Node 16 x → breaks differently everywhere	With Docker: IMAGE: app + Node18 + libX2.1 + config runs identically everywhere
---	--

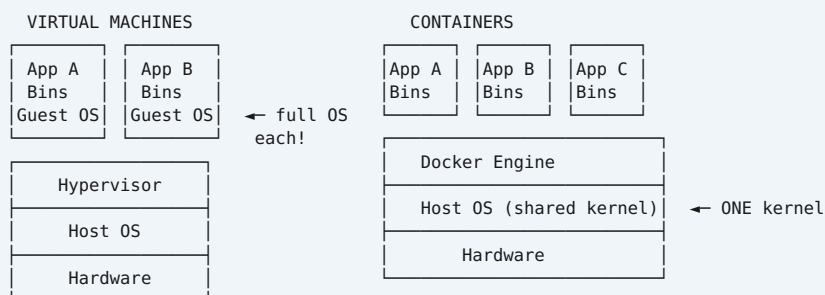
ANALOGY

Shipping containers. Before them, cargo was loaded loose and every port/ship handled it differently — slow and chaotic. The standard steel container meant any crane, ship, or truck could carry it unchanged. Docker does that for software: a standard box any Docker host can run.

7.2 Container vs Virtual Machine INTERVIEW

WHY NOT JUST USE VMS?

A VM virtualizes **hardware** and runs a **full guest OS** — heavy (GBs, slow boot). A container virtualizes the **OS**: all containers share the host's kernel and just isolate the process, filesystem, and network. So containers are megabytes, start in milliseconds, and pack far more densely — at the cost of weaker isolation (shared kernel).



	Virtual Machine	Container
Virtualizes	Hardware	Operating system
Guest OS	Full OS per VM	None — shares host kernel
Size	GBs	MBs
Startup	Seconds-minutes	Milliseconds
Isolation	Strong (separate kernels)	Weaker (shared kernel)
Density	Few per host	Many per host

MEMORY HOOK

VM = a whole house per family (own plumbing, own everything). Container = apartments in one building sharing the plumbing (kernel) — lighter, denser, less isolated.

7.3 IMAGE ≠ CONTAINER (the most important distinction) INTERVIEW

BURN THIS IN

An **image** is a read-only template — a frozen snapshot of the app + dependencies + filesystem. A **container** is a running (or stopped) **instance** of an image, with a writable layer on top. One image → many containers, just like one class → many objects (yes, exactly the OOP relationship from Section 1).

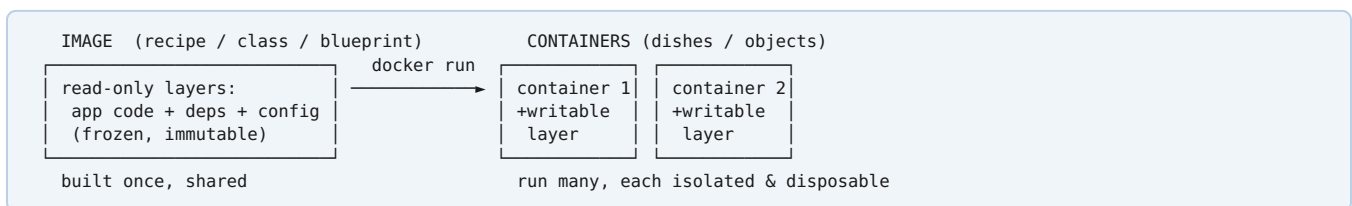


	Image	Container
State	Static, read-only	Running/stopped, has a writable layer
Analogy	Class / recipe / .exe file	Object / cooked dish / running process
How many	Built once	Many from one image
Made by	<code>docker build</code>	<code>docker run</code>

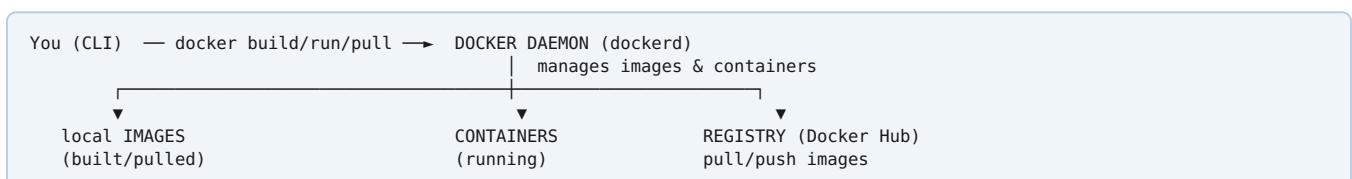
TRAP

"I changed a file in the container; the image is updated." No — changes live only in that container's writable layer and vanish when it's removed (unless you commit a new image or use a volume). The image is immutable. This is why containers are called **disposable** and why persistent data needs volumes (7.6).

QUIZ 7.A

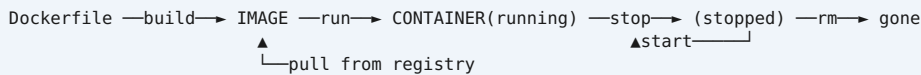
You run the same image three times. How many images and how many containers exist? Do the three share files or have separate ones? (Answer at end.)

7.4 The Docker architecture & ecosystem INTERVIEW



Term	Meaning
Docker Engine	The whole runtime: daemon + API + CLI
Docker daemon (dockerd)	Background process that builds/runs/manages containers & images
Docker CLI	The docker commands you type; talks to the daemon
Docker Hub / registry	Remote store of images (like GitHub for images); pull / push
Dockerfile	The text recipe used to build an image

CONTAINER LIFECYCLE & CORE COMMANDS



Command	Does
<code>docker pull nginx</code>	Download an image from a registry
<code>docker build -t myapp .</code>	Build an image from the Dockerfile here
<code>docker run myapp</code>	Create + start a container from an image
<code>docker ps</code> / <code>docker ps -a</code>	List running / all containers
<code>docker stop</code> / <code>start</code>	Stop / restart a container
<code>docker exec -it <c> bash</code>	Run a command (open a shell) inside a running container
<code>docker logs <c></code>	View a container's output
<code>docker rm <c></code> / <code>docker images</code>	Remove a container / list images

7.5 The Dockerfile INTERVIEW

```

FROM node:18-alpine      # base image to build on
WORKDIR /app             # set working dir inside the image
COPY package*.json ./    # copy dependency manifests first (cache!)
RUN npm install          # install deps (a build-time step → a layer)
COPY . .                 # copy the rest of the source
EXPOSE 3000              # document the port the app listens on
ENV NODE_ENV=production  # set an environment variable
CMD ["node", "server.js"] # default command when a container starts
  
```

Instruction	Purpose
FROM	Base image to start from
WORKDIR	Set the working directory
COPY	Copy files from host into the image
RUN	Execute a command at build time (creates a layer)
CMD	Default command at run time (overridable)
ENTRYPOINT	Fixed executable at run time (args appended)
EXPOSE	Document which port the app uses
ENV	Set environment variables

CMD VS ENTRYPOINT

CMD sets a default that's *entirely replaced* if you pass a command to `docker run`. **ENTRYPOINT** sets a fixed executable; anything you pass to `docker run` becomes its *arguments*. Pattern: `ENTRYPOINT ["python", "app.py"]` + `CMD ["--port", "8000"]` → runs `python app.py --port 8000`, and `docker run img --port 9000` swaps just the args. Rule: ENTRYPOINT = the program, CMD = the default arguments.

LAYERS, CACHE, MULTI-STAGE, .DOCKERIGNORE**HOW IMAGE LAYERS & CACHING WORK**

Each Dockerfile instruction creates a **layer**, stacked read-only. Docker **caches** layers and rebuilds only from the first changed instruction downward. That's why you `COPY package.json` and `RUN npm install` *before* copying all source: if only your source changes, the expensive install layer stays cached. Reorder poorly and every build reinstalls everything.

```

Layer cache: change source code only →
FROM ...                ✓ cached
WORKDIR ...             ✓ cached
COPY package.json       ✓ cached
RUN npm install         ✓ cached ← expensive step SKIPPED (didn't change)
COPY . .                x rebuild ← from here down rebuilds
CMD ...                 x rebuild

```

- **Multi-stage build:** use one big stage to compile/build, then copy only the final artifact into a tiny runtime image — ships a much smaller, safer image (no build tools inside).
- **.dockerignore:** like `.gitignore` — excludes files (`node_modules`, `.git`, secrets) from the build context, keeping builds fast and images clean.

QUIZ 7.B

Your build is slow because `npm install` runs every time even when only app code changed. What's the likely Dockerfile ordering mistake? (Answer at end.)

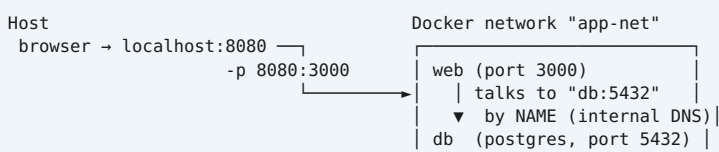
7.6 Volumes & Networking INTERVIEW**WHY VOLUMES**

Containers are disposable — their writable layer dies with them. But databases need data to **survive** restarts. A **volume** is storage managed by Docker that lives *outside* the container's lifecycle, so data persists even when the container is destroyed and recreated.

	Named volume	Bind mount
Managed by	Docker	You (a specific host path)
Use for	Databases, persistent app data	Live-editing source in development

NETWORKING & PORT MAPPING**HOW CONTAINER NETWORKING WORKS**

Containers on the same Docker network can reach each other **by service name** (Docker provides internal DNS). To reach a container from your host/browser, you **publish** a port: `docker run -p 8080:3000` maps host port 8080 → container port 3000. Container-to-container = internal network; host-to-container = port mapping.



7.7 Docker Compose INTERVIEW

WHY COMPOSE

Real apps are multiple containers (web + backend + DB). Running and wiring them by hand with many `docker run -p ... --network ...` commands is error-prone. **Compose** defines the whole multi-container app in one `docker-compose.yml` and starts it all with `docker compose up`.

```
# docker-compose.yml — a real 3-tier app
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    depends_on: [backend]
  backend:
    build: ./backend
    ports: ["5000:5000"]
    environment:
      DATABASE_URL: postgres://user:pass@db:5432/app # 'db' = service name!
    depends_on: [db]
  db:
    image: postgres:16
    volumes: ["pgdata:/var/lib/postgresql/data"] # persist DB data
    environment:
      POSTGRES_PASSWORD: pass
volumes:
  pgdata: # named volume
```

Real architecture, one command (`docker compose up`):

```
Browser → frontend (React) → backend (Node/Express) → db (PostgreSQL)
      :3000                :5000                :5432
      |                    |                    |
      |—— all on one Docker network —|
      | backend reaches db by name "db"
      | db data persists in a named volume
```

QUIZ 7.C

In the Compose file above, how does the backend find the database — by IP address or by name? What provides that? (Answer at end.)

7.8 Docker vs Kubernetes & security (high level) DEEPER

DOCKER VS KUBERNETES

Docker **builds and runs** containers, typically on one host. **Kubernetes (K8s)** is an **orchestrator**: it runs containers across a fleet of machines, handling scaling, self-healing (restart crashed containers), rolling updates, load balancing, and service discovery. Analogy: Docker makes the shipping containers; Kubernetes is the port + fleet + logistics system moving thousands of them. They're complementary, not competitors.

Security basics: don't run as root inside the container; use minimal base images (alpine/distroless) to shrink attack surface; never bake secrets into images (use env/secret managers); scan images for vulnerabilities; set **resource limits** (CPU/memory) so one container can't starve the host.

CONNECTS TO

Image → container is the same one-to-many idea as class → object (§1). Container networking, port mapping, and internal DNS are the **Networks** concepts from §4. Volumes exist because of the container/process **lifecycle** (§3). Compose wires up the exact 3-tier **backend architecture** from §5. Registries (Docker Hub) mirror how GitHub hosts Git repos (§6).

7.9 Confusion Pairs

Pair	Difference
Image vs Container	Read-only template vs running instance (class vs object)
Container vs VM	Shares host kernel (light) vs full guest OS (heavy)
CMD vs ENTRYPOINT	Default replaceable command vs fixed executable + args
Volume vs Bind mount	Docker-managed persistent storage vs specific host path
docker build vs run	Make an image vs start a container
Docker vs Kubernetes	Build/run containers vs orchestrate many across machines
Registry vs image vs container	Store of images vs one image vs a running instance
EXPOSE vs -p	Documents a port vs actually publishes host→container

7.10 Chapter Interview Kit

RAPID RECALL

1. What problem does Docker solve?
2. Image vs container?
3. Container vs VM — one key difference?
4. What is the Docker daemon?
5. What is Docker Hub?
6. build vs run?
7. CMD vs ENTRYPOINT?
8. Why do volumes exist?
9. How do containers talk to each other?
10. What does `-p 8080:3000` do?
11. Why order Dockerfile steps carefully?
12. What is Docker Compose for?

INTERVIEW QUESTIONS

1. Explain image vs container using an analogy.
2. Why is a container lighter than a VM?
3. How does Docker layer caching speed up builds?
4. How do you persist a database's data across container restarts?
5. How do two containers communicate in Compose?
6. What is a multi-stage build and why use it?
7. Difference between Docker and Kubernetes.
8. How would you reduce a Docker image's size?
9. What happens to data written inside a container when it's removed?
10. How do you get a shell inside a running container?

DEEP QUESTIONS

1. Why does changing one line of source sometimes rebuild many layers? (Cache invalidation cascades downward.)
2. Containers share the host kernel — what's the security implication vs VMs? (Weaker isolation.)
3. Why are containers called "disposable" and how does that shape app design? (Stateless; externalize state to volumes/DB.)
4. Why put `npm install` before copying source in a Dockerfile? (Cache the expensive step.)
5. When do you actually need Kubernetes vs plain Docker/Compose? (Multi-host scale, self-healing, rolling deploys.)

TRAP QUESTIONS

1. "An image and a container are the same." (No — template vs instance.)
2. "A container is a lightweight VM." (Loosely; no guest OS — shares kernel.)

3. "Data in a container is safe across restarts." (No — needs a volume.)
4. "EXPOSE publishes the port to the host." (No — that's `-p`.)
5. "Docker replaces Kubernetes." (No — complementary.)

EXPLAIN-IT-ALLOUD

- Explain image vs container to a junior dev.
- Explain container vs VM with the layer diagram.
- Explain how Docker layer caching works and how to exploit it.
- Explain how a web + backend + DB app is wired with Compose.

30-SECOND EXPLANATIONS

DOCKER IN 30S

Docker packages an app with all its dependencies and environment into an image, so it runs identically anywhere Docker is installed — killing "works on my machine." An image is a read-only template; a running instance of it is a container. Containers share the host kernel, so they're far lighter and faster than VMs.

IMAGE VS CONTAINER IN 30S

An image is a frozen, read-only snapshot of your app plus its filesystem and dependencies, built from a Dockerfile. A container is a live, isolated instance of that image with a thin writable layer on top. One image can spawn many containers, exactly like one class can create many objects — and container changes vanish on removal unless you use a volume.

ONE PICTURE FOR THE WHOLE CHAPTER

Write a Dockerfile (recipe) → build an image (frozen meal kit) → run containers (cooked, disposable dishes) that share the host kernel, talk over a Docker network by name, and keep important data in volumes. Compose wires several containers into one app; Kubernetes runs that app across a whole fleet.

Chapter 7 — Quiz Answers

ANSWERS

7.A One image, three containers. They all share the same read-only image layers (Docker doesn't copy them), but each container has its own separate writable layer, so changes in one don't affect the others.

7.B You're probably running `COPY . .` (all source) *before* `RUN npm install`. Any source change invalidates the cache for that `COPY` and everything after it, forcing a reinstall. Fix: `COPY package*.json ./` then `RUN npm install` first, and `COPY . .` afterward, so the install layer stays cached when only source changes.

7.C By name — the backend connects to `db:5432` using the service name `db`, not an IP. Docker's built-in DNS on the shared network resolves service names to the right container, so you never hardcode changing IP addresses.

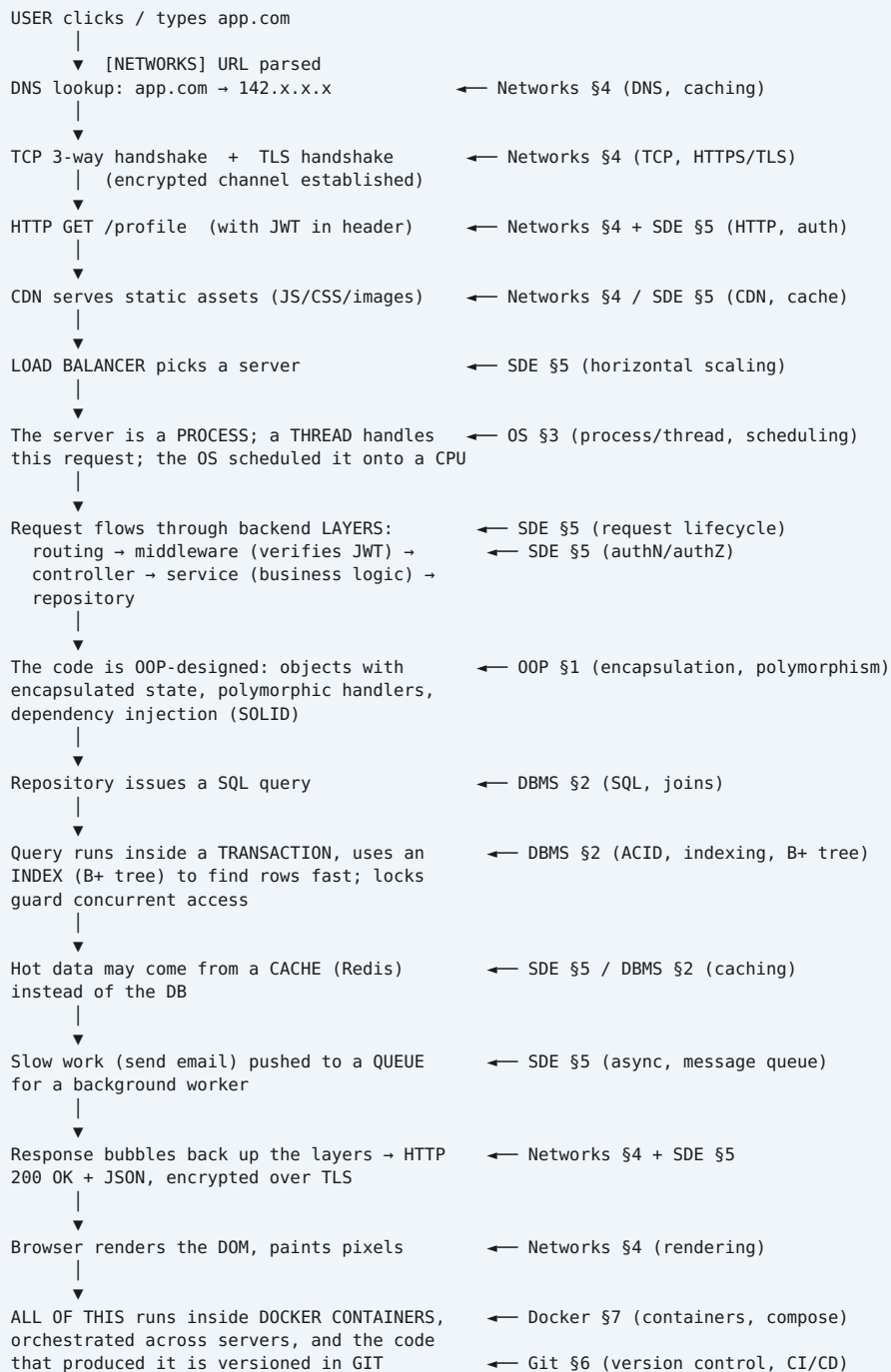
SECTION 8

How It All Fits Together

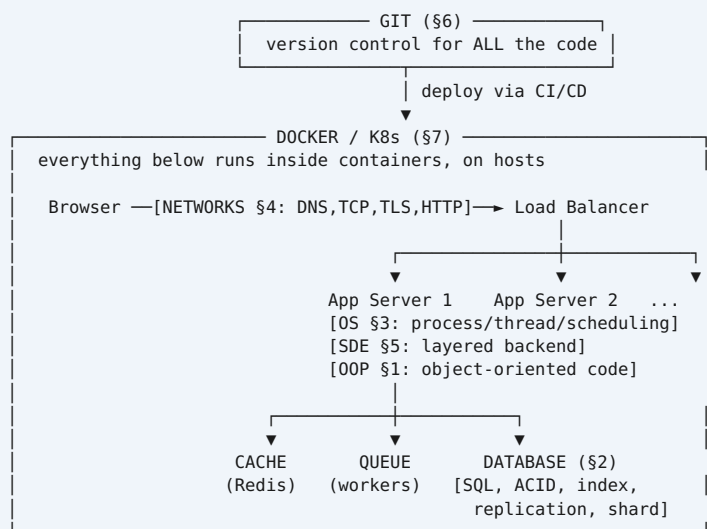
Strong candidates don't answer topics in isolation — they show how the pieces interlock in a real system. This section is the glue. If you can narrate one request from click to pixels and name which subject owns each step, you sound like someone who has actually built software.

8.1 One request, all seven subjects INTERVIEW

A user opens your web app and loads their profile. Here's the full journey, tagged by subject:



8.2 The system map



8.3 Recurring themes across subjects DEEPER

The same handful of ideas appear again and again under different names. Recognizing them is what "getting it" feels like.

Theme	Where it shows up
Concurrency & locking	OS: mutex/semaphore, deadlock (\$3). DBMS: transactions, shared/exclusive locks, deadlock (\$2). Same problem (shared state + parallel access), same tools (locks), same failure (deadlock).
Caching	OS: TLB, CPU cache, page cache (\$3). Networks: DNS cache, CDN, HTTP cache (\$4). DBMS: buffer pool, query cache (\$2). SDE: Redis (\$5). Always: trade freshness for speed; invalidation is the hard part.
Layering / abstraction	Networks: OSI/TCP-IP layers (\$4). SDE: controller/service/repository (\$5). OOP: abstraction, interfaces (\$1). Each layer hides the one below.
Indirection / pointers	OS: page tables, virtual→physical (\$3). OOP: references, vtables (\$1). DBMS: indexes point to rows (\$2). Git: branches/HEAD are pointers (\$6). "One more level of indirection solves everything."
One-to-many templates	OOP: class → objects (\$1). Docker: image → containers (\$7). Exactly the same relationship.
Trees for fast lookup	DBMS: B+ tree indexes (\$2). DNS: hierarchical tree (\$4). Filesystems: directory trees (\$3). Git: tree objects (\$6).
Statelessness for scale	Networks: HTTP is stateless (\$4). SDE: stateless services scale horizontally, JWT is stateless auth (\$5). Docker: disposable containers (\$7). State is pushed out to DBs/volumes/caches.
Snapshots & logs for durability	DBMS: write-ahead log (\$2). Git: commit snapshots (\$6). OS: swap/checkpoints (\$3). Record-then-apply so you survive crashes.

THE UNIFYING SENTENCE

Almost every CS concept in this guide exists to manage one of three things: **complexity** (layering, abstraction, OOP, modularity), **concurrency** (locks, transactions, isolation, threads), or **scale/latency** (caching, indexes, replication, sharding, CDNs, containers). Name which one a topic serves and you understand why it exists.

8.4 Cross-subject interview questions INTERVIEW

- Trace a request from browser to database and back, naming every subject involved. (The §8.1 journey.)
- Deadlocks appear in both OS and DBMS — are they the same problem? (Yes: cyclic wait on locked resources; both detect/abort or prevent via ordering.)
- Caching appears at many layers — name four and the shared risk. (TLB, CPU cache, DNS/CDN, HTTP, DB buffer, Redis; risk = staleness/invalidation.)

4. How is a class→object like an image→container? (Immutable template → many live instances.)
5. Why do both HTTP and horizontally-scaled services favor statelessness? (Any node handles any request; no shared session pinning.)
6. A page is slow. Walk through where the bottleneck could be at each layer. (DNS, network latency, TLS, LB, app CPU/threads, N+1 query, missing index, DB lock contention, cold cache, container limits.)
7. Both Git and databases talk about "logs" and "commits" — connect the ideas. (Append-only history; durability; atomic apply.)
8. How does an index (DBMS) relate to a page table (OS)? (Both are lookup structures turning a key into a location fast.)

INTERVIEW POWER MOVE

When asked about any single topic, land the answer, then add one sentence connecting it outward: "...and this is the same locking problem the OS solves with mutexes." That one bridge signals depth more than any memorized definition.

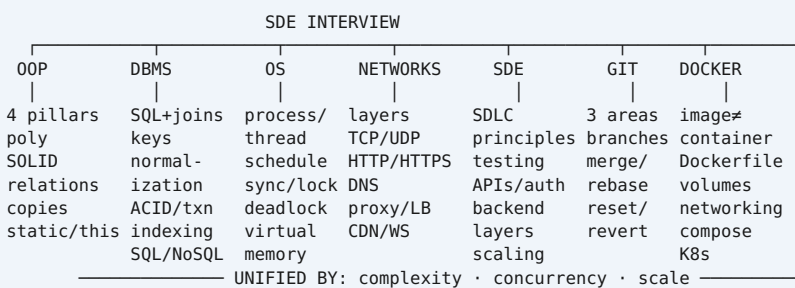
SECTION 9

Final Revision

The consolidation chapter. Mind maps to see each subject at a glance, a one-day and one-week plan for the run-up, a 150+ question bank sorted by difficulty, and a full mixed mock interview. Use this after you've read the subject chapters at least once.

9.1 Mind Maps

Master SDE Interview Map



OOP

OOP — goal: make large code safe to change

- └ Class / Object / State / Behavior
- └ Constructor / Destructor (lifetime, memory)
- └ FOUR PILLARS
 - └ Encapsulation (hide+protect data)
 - └ Abstraction (hide complexity)
 - └ Inheritance (is-a, reuse)
 - └ Polymorphism (compile:overload / runtime:override-vtable)
- └ Abstract class vs Interface
- └ Relationships: association / aggregation / composition (has-a vs is-a)
- └ static · this · access modifiers · shallow vs deep copy
- └ SOLID (S R P, O/C, Liskov, Interface seg, Dependency inv)

DBMS

DBMS — correctness · concurrency · durability · speed

- └ Why not files: query, concurrency, integrity, crash safety
- └ SQL: SELECT-FROM-WHERE-GROUP BY-HAVING-ORDER BY (exec order!)
 - └ aggregates · subqueries · CTE · CASE
 - └ JOINS: inner / left / right / full / self
- └ Keys: primary / foreign / candidate / composite + referential integrity
- └ Normalization: 1NF→2NF→3NF→BCNF (key, whole key, nothing but the key)
- └ Transactions: ACID · isolation levels · dirty/non-repeatable/phantom
 - └ locks (S/X) · deadlock
- └ Indexing: B+ tree · clustered vs non-clustered · read/write tradeoff
 - └ SQL vs NoSQL · replication · sharding · pooling

Operating Systems

OS — juggle many programs on shared hardware; keep the illusion

- └ Kernel · user mode vs kernel mode · system calls
- └ Process (isolated) vs Thread (shared memory) · PCB · context switch
- └ Scheduling: FCFS · SJF · SRTF · Round Robin · Priority (+ metrics)
- └ Synchronization: race · critical section · mutex vs semaphore · monitor
- └ Deadlock: 4 conditions · prevent/avoid/detect/recover · Banker's
- └ Memory: virtual mem · paging (page/frame/offset) · page table · TLB
 - └ page fault · demand paging · thrashing · replacement (FIFO/LRU/OPT)
- └ "What happens when you run a program?"

Computer Networks

NETWORKS — nested envelopes; layers each do one job

- └ IP vs MAC vs Port vs Packet · LAN/WAN
- └ Models: OSI(7) / TCP-IP(4): App-Transport-Network-Link
- └ Transport: TCP (handshake, ACK, retransmit, flow, congestion) vs UDP
- └ HTTP: methods · status codes · headers · cookies/sessions · caching · REST
- └ HTTPS/TLS: cert+CA (auth) · asymmetric-symmetric key · encryption
- └ DNS: name-IP hierarchy + caching → "type google.com" journey
- └ IPv4/6 · NAT · DHCP · routing · proxy/reverse proxy · LB · CDN · WebSockets

Software Engineering / SDE

SDE — process · design quality · safe change

- └ SDLC: requirements-design-build-test-deploy-maintain
- └ Methodologies: Waterfall vs Agile (Scrum sprints / Kanban flow)
- └ Principles: modularity · high cohesion / low coupling · DRY/KISS/YAGNI · SOLID
- └ Testing pyramid: unit → integration → e2e (+ mock vs stub)
- └ APIs: REST · JSON · authN vs authZ · sessions vs JWT
- └ Backend layers: routing-middleware-controller-service-repository-DB
 - └ caching · queues · async/background · logging · monitoring · config/env
- └ Scaling: vertical vs horizontal · LB · replica · shard · queue · CDN

Git

GIT — time machine + merge engine (distributed)

- └ 3 areas: working dir → (add) → staging → (commit) → repository
- └ Everyday: init/clone/status/add/commit/log/diff/push/pull/fetch
- └ fetch vs pull (download vs download+merge)
- └ Branches: branch/switch/merge/rebase · conflicts
- └ merge vs rebase (preserve vs rewrite; golden rule)
- └ reset vs revert · stash · cherry-pick · .gitignore · tags · HEAD/origin
- └ Commit internals: blob→tree→commit(+parent), hashed snapshots · Git vs GitHub

Docker

DOCKER — package app+env; runs identically everywhere

- └ Problem: "works on my machine"
- └ Container vs VM (shared kernel, light vs full OS, heavy)
- └ IMAGE ≠ CONTAINER (template vs instance; class vs object)
- └ Engine/daemon/CLI · Docker Hub/registry · lifecycle
- └ Commands: pull/build/run/ps/stop/start/exec/logs/rm/images
- └ Dockerfile: FROM/WORKDIR/COPY/RUN/CMD/ENTRYPOINT/EXPOSE/ENV
 - └ layers+cache · multi-stage · .dockerignore (CMD vs ENTRYPOINT)
- └ Volumes (persist) · networking (name-based DNS, -p port map)
- └ Compose (multi-container) · Docker vs Kubernetes

9.2 One-Day Revision Sheet (highest-value only)

IF YOU ONLY HAVE HOURS, KNOW THESE COLD

OOP: 4 pillars with the "why," overloading vs overriding + vtable, is-a vs has-a, SOLID one-liners.

DBMS: SELECT execution order, WHERE vs HAVING, 4 joins, ACID, isolation anomalies, why/when indexes (B+ tree), normalization 1→3NF.

OS: process vs thread, context switch cost, race condition + mutex vs semaphore, 4 deadlock conditions, virtual memory + paging + page fault, "what happens when you run a program."

Networks: TCP vs UDP, 3-way handshake (why 3), HTTP methods + status families, 401 vs 403, HTTPS/TLS (cert + asymmetric→symmetric), "type google.com" journey.

SDE: cohesion/coupling, test pyramid + mock vs stub, authN vs authZ, backend layers, horizontal vs vertical scaling, N+1 problem, caching + queues.

Git: 3 areas, fetch vs pull, merge vs rebase + golden rule, reset vs revert, Git vs GitHub, what a commit is.

Docker: image vs container, container vs VM, Dockerfile basics, CMD vs ENTRYPOINT, volumes (persistence), layer caching, Compose, Docker vs K8s.

9.3 One-Week Revision Plan

Day	Focus	Do
Day 1	OOP	Read §1, do all quizzes + Rapid Recall aloud, sketch the vtable and relationships diagrams from memory.
Day 2	DBMS	Read §2, write the 4 joins + a normalize-to-3NF example by hand, do the practical SQL problems, recite ACID + isolation table.
Day 3	OS	Read §3, compute FCFS/SJF/RR by hand, redo the paging + page-replacement numbers, narrate "what happens when you run a program."
Day 4	Networks	Read §4, draw the 3-way handshake + TLS + "type google.com" from memory, recite TCP vs UDP and status codes.
Day 5	SDE + Git + Docker	Read §5–§7, draw backend layers + Compose 3-tier, recite merge vs rebase and image vs container.
Day 6	Connections + weak spots	Read §8, narrate the full request journey, re-drill any quiz you failed, redo Explain-It-Aloud prompts.
Day 7	Mock + question bank	Do the §9.5 mock under time pressure, then sweep the §9.4 bank; mark anything shaky for a final morning pass.

HOW TO REVISE (NOT JUST READ)

For each topic: close the guide and say the 30-second explanation aloud, then draw the key diagram from memory, then check. If you can teach it out loud and draw it, you own it. Re-reading feels productive but builds far weaker memory than recall.

9.4 Interview Question Bank (150+)

Organized by difficulty. Try each aloud before checking yourself against the relevant chapter. Most answers live in the section noted; the toughest have short pointers.

Beginner FOUNDATION

1. What problem does OOP solve that procedural code struggles with?
2. Difference between a class and an object?
3. What is encapsulation and what breaks without it?
4. What is abstraction? How does it differ from encapsulation?
5. Name the four pillars of OOP.
6. What is inheritance and what relationship does it model?
7. What does a constructor do?
8. What does `static` mean on a field or method?

9. What is a database and why not just use files?
10. Difference between DBMS and RDBMS?
11. What are rows, columns, and a schema?
12. What does a `WHERE` clause do?
13. What is a primary key?
14. What is a foreign key?
15. What is an index, in one sentence?
16. What does `SELECT DISTINCT` do?
17. What is an operating system responsible for?
18. Difference between a program and a process?
19. Difference between a process and a thread?
20. What is a deadlock (informally)?
21. What is RAM vs disk, in terms of speed?
22. What is an IP address? A port?
23. Difference between an IP address and a MAC address?
24. What is a packet?
25. What is DNS?
26. What does HTTP stand for and do?
27. Name three HTTP methods and what they do.
28. What does a 404 status code mean? A 500?
29. What does HTTPS add over HTTP?
30. What is TCP vs UDP at a high level?
31. What is an API?
32. What is JSON?
33. What is the difference between authentication and authorization?
34. What are the phases of the SDLC?
35. What is the difference between Waterfall and Agile?
36. What is a unit test?
37. What does DRY stand for?
38. What problem does version control solve?
39. Name Git's three areas.
40. What is the difference between `git add` and `git commit`?
41. What is the difference between Git and GitHub?
42. What is a branch in Git?
43. What problem does Docker solve?
44. What is the difference between an image and a container?
45. What is a Dockerfile?
46. What is Docker Hub?
47. What does `docker run` do vs `docker build`?
48. What is a container vs a virtual machine, in one line?

Intermediate Interview

1. Explain overloading vs overriding with examples.
2. What makes runtime polymorphism work internally?
3. Abstract class vs interface — when do you pick each?
4. Difference between composition and inheritance? Why prefer composition?
5. Explain association vs aggregation vs composition.
6. Shallow vs deep copy — when does the difference bite you?
7. State each SOLID principle with a one-line example.
8. What is the Liskov Substitution Principle? Give a violation.
9. Explain the execution order of a SQL `SELECT`.
10. `WHERE` vs `HAVING` — when do you use each?
11. Explain `INNER` vs `LEFT` vs `FULL OUTER JOIN`.
12. What is a self join and when is it useful?
13. Write a query for the second-highest salary.

14. How would you find duplicate rows?
15. Explain 1NF, 2NF, 3NF with the anomaly each removes.
16. State ACID and what each guarantee prevents.
17. Explain dirty read, non-repeatable read, and phantom read.
18. What are the isolation levels and their trade-off?
19. Why is a B+ tree index fast for both lookups and ranges?
20. Clustered vs non-clustered index?
21. Why not create an index on every column?
22. SQL vs NoSQL — how do you choose?
23. Replication vs sharding?
24. Explain user mode vs kernel mode and why system calls are costly.
25. What is in a PCB and what happens during a context switch?
26. Compute average waiting time for given processes under FCFS and SJF.
27. Why does Round Robin favor responsiveness over average waiting time?
28. Explain a race condition and fix it with a mutex.
29. Mutex vs semaphore — when do you use a counting semaphore?
30. State the four deadlock conditions and one way to break each.
31. Explain virtual memory and how a page table works.
32. Compute page number and offset for a given address and page size.
33. What is a page fault and what does the OS do on one?
34. FIFO vs LRU page replacement — trace a short reference string.
35. What is thrashing and how do you fix it?
36. Explain the TCP 3-way handshake and why it's three steps.
37. Flow control vs congestion control?
38. Give a real use case for TCP and one for UDP.
39. Explain the five HTTP status code families.
40. 401 vs 403 — and how it maps to authN/authZ.
41. How do cookies and sessions make stateless HTTP feel stateful?
42. Walk through the TLS handshake at a high level.
43. Why does TLS use both asymmetric and symmetric encryption?
44. Describe DNS resolution and where caching happens.
45. Proxy vs reverse proxy vs load balancer?
46. When would you use WebSockets instead of HTTP polling?
47. Explain high cohesion and low coupling with an example.
48. Explain the test pyramid and why it's shaped that way.
49. Mock vs stub — what does each verify?
50. Sessions vs JWT — trade-offs?
51. Walk a request through the backend layers.
52. What is the N+1 query problem and how do you fix it?
53. Why and when would you introduce a message queue?
54. Horizontal vs vertical scaling — trade-offs?
55. fetch vs pull in Git?
56. merge vs rebase — and the golden rule of rebase?
57. reset vs revert — which for a pushed commit?
58. What are the three reset modes?
59. How do you resolve a merge conflict?
60. What is `git stash` and `git cherry-pick` for?
61. Explain CMD vs ENTRYPOINT in a Dockerfile.
62. Why do Docker volumes exist?
63. How do two containers communicate in Docker Compose?
64. How does Docker layer caching speed up builds?
65. What does `-p 8080:3000` do?

Advanced DEEPER

1. Walk through a virtual method call at the vtable/vptr level.

2. Why is every Java method virtual by default but C++ requires `virtual` ? Trade-off?
3. If interfaces support default methods now, what still separates them from abstract classes?
4. Give a case where inheritance "works" but is the wrong design.
5. Why can overloading be resolved at compile time but overriding cannot?
6. What breaks if you override `equals()` but not `hashCode()` ?
7. Explain the diamond problem and how languages resolve it.
8. Why can't `WHERE` use aggregate functions but `HAVING` can?
9. Why does REPEATABLE READ prevent non-repeatable reads but not always phantoms?
10. How does the DBMS guarantee durability across a crash? (WAL)
11. When will the query optimizer ignore an existing index?
12. Explain the leftmost-prefix rule for composite indexes.
13. When would you deliberately denormalize, and what do you give up?
14. How are OS synchronization and DB transactions the same problem?
15. Why does too-frequent context switching reduce throughput despite each switch being fast?
16. Why isn't SJF used directly in real schedulers despite optimal average waiting?
17. What is Belady's anomaly and which policy suffers it?
18. How can multithreading speed up I/O-bound work on a single core?
19. Why does congestion control benefit everyone even though it slows an individual sender?
20. What does the CA/certificate add on top of TLS encryption, and why is it essential?
21. What parts of an HTTPS request are visible on the network vs encrypted?
22. HTTP is stateless yet apps feel stateful — where does the state actually live?
23. Why can NAT complicate peer-to-peer connections?
24. Why does over-applying DRY sometimes make code worse?
25. Why build stateless services for horizontal scaling?
26. JWT is stateless — what problem does that create for logout/revocation, and the mitigation?
27. Caching's hardest problem is invalidation — why?
28. Why does rebasing create new commit hashes, and why does that matter on shared branches?
29. Git stores snapshots, not diffs — how does it stay space-efficient?
30. Why is a Git commit hash derived from content and parent?
31. How does an OS page table relate conceptually to a DB index?
32. Why are containers "disposable," and how does that shape application design?
33. Containers share the host kernel — what's the security implication vs VMs?
34. When do you actually need Kubernetes over plain Docker/Compose?
35. Why order `npm install` before copying source in a Dockerfile?
36. How would you debug a slow web page, layer by layer, from DNS to DB?
37. Design a URL shortener at a high level: what components and why?
38. How would you scale a read-heavy service to millions of users?
39. How would you make a payment endpoint safe to retry? (Idempotency.)
40. Two services must not double-charge a card during a race — how? (Locks/idempotency keys/transactions.)
41. How do CDNs, caches, and read replicas together reduce load on the origin DB?
42. Explain how a request that "just loads a profile" touches all seven subjects.
43. What's the difference between optimistic and pessimistic locking? (Deeper DB concurrency.)
44. How does connection pooling improve DB performance under load?
45. Why might adding more threads slow a CPU-bound program down?
46. Explain eventual consistency and when it's acceptable.
47. How would you prevent deadlocks in code acquiring multiple locks? (Lock ordering.)
48. What is a memory leak in a garbage-collected language, and how does it happen? (Lingering references.)
49. What is the "thundering herd" problem with caching, and how do you mitigate it? (Cache stampede → locks/staggered TTL.)
50. How do you safely roll back a bad deployment? (Blue-green/canary, revert, DB migration care.)
51. Explain idempotency across GET/PUT/DELETE/POST and why retries depend on it.
52. How would you store passwords safely? (Hash + salt, bcrypt/argon2 — never plaintext/encryption.)

9.5 Final Self-Test — Mixed Mock Interview

Simulate a real 45-minute technical screen. Answer aloud, don't peek. A concise answer key follows at the very end of the guide (\$9.6) — grade yourself after finishing all questions.

ROUND 1 — WARM-UP (OOP + FUNDAMENTALS)

M1.

Explain the four pillars of OOP, and for each, say the problem it solves.

M2.

Overloading vs overriding — and what makes overriding work at runtime?

M3.

"Prefer composition over inheritance" — why? Give an example where inheritance is wrong.

ROUND 2 — DATABASES

M4.

Why is `HAVING` different from `WHERE`? Tie it to SELECT execution order.

M5.

Write a query returning each customer and their total spend, only for customers who spent over 1000.

M6.

Explain ACID using a bank transfer, then explain which isolation level you'd choose and why.

M7.

Why is an index fast, why not index everything, and what's a B+ tree's advantage for ranges?

ROUND 3 — OPERATING SYSTEMS

M8.

Process vs thread — explain to a junior, then say why threads need synchronization.

M9.

Given P1(burst 4), P2(burst 2), P3(burst 6) arriving at 0, compute average waiting time under SJF.

M10.

Explain virtual memory, paging, and what happens on a page fault.

M11.

Walk me through everything that happens when you run `./app`.

ROUND 4 — NETWORKS

M12.

TCP vs UDP — and which would you pick for a video call, and why?

M13.

Explain the 3-way handshake and why it isn't two messages.

M14.

How does HTTPS keep data private *and* prove you're talking to the real server?

M15.

Walk me through what happens, end to end, when you type `google.com`.

ROUND 5 — SDE & DESIGN**M16.**

Explain cohesion and coupling and why you want high one and low the other.

M17.

Authentication vs authorization, and how JWT and the 401/403 codes fit.

M18.

A request to generate a big report takes 90 seconds. How do you design the endpoint?

M19.

How would you scale a read-heavy API to millions of users? Name the pieces.

M20.

What's the N+1 query problem, how would you spot it, and how do you fix it?

ROUND 6 — GIT & DOCKER**M21.**

Explain the three Git areas and what `add` / `commit` move between them.

M22.

merge vs rebase — when each, and the golden rule?

M23.

You pushed a secret to `main`. What do you do?

M24.

Image vs container — explain with an analogy, and why data disappears without a volume.

M25.

Your Docker build reinstalls dependencies on every code change. Diagnose and fix.

ROUND 7 — THE CONNECTING QUESTION**M26.**

A user loads their dashboard and it's slow. Walk through, layer by layer from DNS to database, where the bottleneck could be and how you'd investigate each.

M27.

Name three places caching appears across these subjects and the one risk they all share.

M28.

Deadlocks appear in both OS and DBMS — are they the same problem? Justify.

9.6 Mock Interview — Answer Key

CONCISE ANSWERS (GRADE YOURSELF)

M1 Encapsulation (protect state/invariants), Abstraction (hide complexity behind a clean interface), Inheritance (reuse via is-a), Polymorphism (one interface, many behaviors → extensible, no type if/else). All serve safe change. **M2** Overloading = same name/different params, resolved at compile time; overriding = subclass redefines an inherited method, resolved at runtime via the vtable/vptr.

M3 Inheritance couples tightly and can leak base behavior; composition is flexible (has-a). Wrong case: Square extends Rectangle (Liskov) or Stack extends Vector.

M4 WHERE filters rows before grouping (no aggregates); HAVING filters groups after GROUP BY (aggregates allowed) — because execution is FROM→WHERE→GROUP BY→HAVING→SELECT. **M5** `SELECT customer_id, SUM(amount) FROM Orders GROUP BY customer_id HAVING SUM(amount) > 1000;` (join to Customers for names). **M6** Atomic/Consistent/Isolated/Durable via a transfer; choose SERIALIZABLE (or REPEATABLE READ) for money to avoid dirty/phantom reads — correctness over throughput. **M7** Index = sorted lookup ($O(\log n)$) vs full scan; not everything because writes must update every index + storage cost; B+ tree leaves are linked/sorted so ranges scan sequentially.

M8 Process = isolated memory; thread = shares process memory, cheaper. Shared memory → race conditions → need locks. **M9** SJF order P2(2), P1(4), P3(6): waits 0, 2, 6 → avg = $8/3 \approx 2.67$. **M10** Each process gets a virtual address space mapped to frames via a page table (offset unchanged); a page fault means the page isn't in RAM — OS loads it from disk into a frame, updates the table, resumes.

M11 source→compile→executable→fork/exec→process+PCB→virtual address space→scheduler→CPU→page faults load pages→syscalls trap to kernel for I/O→exit frees resources.

M12 TCP reliable/ordered/connection; UDP fast/best-effort. Video call → UDP (late data is useless; drop > delay). **M13** SYN, SYN-ACK, ACK — both sides must confirm send+receive and agree sequence numbers; two messages confirm only one direction. **M14** Server sends a CA-signed certificate (identity); client verifies the chain, then they agree a symmetric session key via asymmetric crypto and encrypt all traffic (confidentiality + integrity + authentication). **M15** parse URL→DNS→TCP handshake→TLS handshake→HTTP GET→server (LB→app→cache/DB)→response (CDN for static)→browser renders.

M16 Cohesion = a module focused on one job (want high); coupling = interdependence (want low) → change one module without touching others. **M17** AuthN = who you are (401 if missing); AuthZ = what you may do (403 if forbidden); JWT carries signed claims so any server verifies statelessly. **M18** Don't block the request: accept it, return 202, enqueue a job for a background worker, notify on completion. **M19** LB + stateless app servers (horizontal), cache (Redis), read replicas, CDN for static, sharding if data huge, queues for async. **M20** $N+1 = 1$ query for a list + 1 per item; spot via query logs/count; fix with a JOIN or eager loading.

M21 Working dir (edit) → `add` → staging (index) → `commit` → repository (history). **M22** Merge preserves history with a merge commit (use on shared branches); rebase rewrites into a linear history (use on local unpushed work); never rebase shared/pushed commits. **M23** Revert forward + rotate/invalidate the leaked secret + purge history (filter-repo) and coordinate the team; reset would rewrite pushed history. **M24** Image = read-only template; container = running instance (class vs object). Container writable layer is discarded on removal → use a volume to persist. **M25** `COPY . .` is before `RUN npm install`, so any code change busts the cache; move dependency install before copying full source.

M26 DNS latency → TLS/handshake → LB → app CPU/thread starvation → N+1 or missing index → DB lock contention → cold cache → container CPU/memory limits; investigate each with timing/metrics/EXPLAIN/logs. **M27** e.g. CPU cache/TLB (OS), DNS/CDN/HTTP cache (Networks), DB buffer pool/Redis (DBMS/SDE); shared risk = staleness/invalidation. **M28** Yes — both are a cycle of parties each holding a resource and waiting for another's; both are handled by prevention (lock ordering) or detection + aborting a victim.

End of guide. You rebuilt the fundamentals from the "why" up. Now close the book, pick any 30-second explanation, and say it out loud. If you can teach it, you're ready.